



ZÁRÓDOLGOZAT

Készítették:

Bihari Csaba – Gaál Gergely Benjámin-Molnár Márkó 2/14E

Konzulens:

Farkas Zoltán

Miskolc

2025.

Miskolci SZC Kandó Kálmán Informatikai Technikum

Miskolci Szakképzési Centrum

SZOFTVERFEJLESZTŐ- ÉS TESZTELŐ SZAK

Dokumentáció

Warehouse webapplikáció

Bihari Csaba – Gaál Gergely Benjámin-Molnár Márkó

Konzulens: Farkas Zoltán

2024-2025

Tartalomjegyzék

Tartalom

1.	Témaválasztás indoklás	1
2.	A felhasznált technológiák bemutatása	2
2.1.	Adatbázis – MySQL.....	2
2.2.	Backend – ASP.net core 8, Entity Framework.....	2
2.3.	Frontend – React, React-Router-Dom, Vanta, JWT	3
2.4.	Verziókezelés – Git/GitHub	4
3.	Felhasznált fejlesztői környezetek bemutatása.....	5
3.1.	Adatbázis - XAMPP (adatbázis host-olás)	5
3.2.	Backend – Visual studio	7
3.3.	Frontend – Visual studio Code	8
4.	Felhasznált projektmanagement eszközök	9
4.1.	Trello	9
4.2.	Discord	10
5.	A szoftver bemutatása.....	10
5.1.	Frontend, webes felhasználói felület működése	10
5.2.	Backend.....	29
5.3.	Adatbázis.....	34
6.	Irodalomjegyzék	38
7.	Ábra jegyzék.....	38
8.	Mellékletek	39
8.1.	Github link.....	39
8.2.	Trello link.....	39

1. Témaválasztás indoklás

Személyes munkatapasztalataink alapján ezt a témát találtuk a legideálisabbnak a projektünk elkészítéséhez, amihez a mindennapokban előforduló észrevételek és hiányosságok inspiráltak minket. A modern vállalkozások működésében a készletgazdálkodás és az erőforrások hatékony kezelése kiemelkedő szerepet játszik. A raktárak, mint logisztikai központok, kulcsszereplői a termelés és az értékesítés közötti kapcsolatnak. A digitális megoldások, mint a raktárkezelő rendszerek (Warehouse Management Systems – WMS), lehetővé teszik a készletek nyilvántartását, mozgásainak követését és a leltározási folyamatok automatizálását. A raktárkezelés célja, hogy minimalizálja a készletek kezelése során fellépő hibákat, biztosítsa a termékek gyors és pontos elérhetőségét, valamint optimalizálja a tárolási helyek kihasználtságát.

A projektünk célja egy webes alapú raktárkezelő rendszer létrehozása, amely segíti a raktározási folyamatok hatékonyabb kezelését, átláthatóságát és automatizálását. A rendszer képes különböző termékek nyilvántartására, azok állapotának követésére, valamint felhasználói jogosultságkezelésre is.

A fejlesztéshez modern technológiákat alkalmaztunk:

- **Frontend:** React (modern, komponens alapú felhasználói felület)
- **Backend:** ASP.NET Core Web API (megbízható és skálázható REST API)
- **Adatbázis:** MySQL (strukturált adattárolás és relációk támogatása)
- **Authentikáció:** JWT tokenekkel támogatott szerep alapú jogosultságkezelés
- **ORM (Object-Relational Mapping):** Entity Framework Core

A rendszer három felhasználó jogosultsági szinttel dolgozik:

- **User:** új anyagot rögzíthet, de nem törölhet és módosíthat
- **Super User:** az anyagokat módosíthatja és törölheti
- **Admin:** minden funkcióhoz teljes hozzáféréssel rendelkezik + felhasználókat módosíthat és törölhet

2. A felhasznált technológiák bemutatása

2.1. Adatbázis – MySQL

A MySQL egy széles körben használt nyílt forráskódú relációs adatbázis-kezelő rendszer (RDBMS), amely jól illeszkedik a készletkezelő rendszerhez, mivel képes hatékonyan tárolni a strukturált adatokat és kezelni a relációkat. Az adatok biztosítják a rendszer számára a szükséges információkat, például a termékek nyilvántartását, a felhasználói adatokat és a raktárak helyszíneit. Először 1995-ben vezették be, és azóta a világ egyik legnépszerűbb adatbázisrendszerévé vált.



2.2. Backend – ASP.net core 8, Entity Framework

Az ASP.net keretrendszert a .Net fejlesztőkörnyezet része, amely lehetővé teszi különféle webes alkalmazások és szolgáltatások gyors és hatékony fejlesztését. A keretrendszer nyílt forráskódú, multiplatform (fut Windows, Linux és Mac platformokon), különféle eszközöket, nyelveket (C#, F#) és függvény könyvtárakat tartalmaz. Kiadási ciklusa éves, egy hosszabb és egy rövidebb támogatási periódus váltja egymást. Jelenleg a 9-es kiadás a legfrissebb, ez egy rövidebb támogatási periódussal rendelkező kiadás. A rendszer moduláris, a modulok megfelelő alkalmazásának kombinációjával az alábbi applikáció feladatokat képes ellátni többek között:

Applikáció típus	Feladat(hozzávaló eszköz)
Web applikáció	új, szerver oldali web UI fejlesztés (Razor)
Web applikáció	MVC architektúra alapú web applikáció fejlesztés
Web applikáció	kliens oldali UI fejlesztés (Blazor)
WEB API	RESTful HTTP szolgáltatások (web API, minimal API)
gRPC applikáció	contact-first szolgáltatások Protocol Bufferek használatával
valós idejű applikáció	kétirányú kommunikáció megvalósítása szerver és kliens között

Tartalmaz továbbá például biztonsági eszközöket autentikációra, autorizációra, CORS-ra {Cross origin resource sharing}, valamint tartalmaz még egyéb, korszerű web alkalmazások fejlesztése során felmerülő feladatok elvégzésére szolgáló eszközöket.

Bár nem az ASP.NET részre, az adatok adatbázisból történő elérését Entity Framework-kön keresztül képes kezelni, amely egy ORM szoftver és szintén a .NET környezet része.

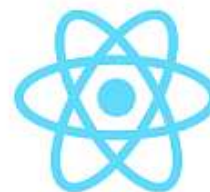
A feladat során felhasznált elemei az ASP.NET keretrendszernek:

- WEB API applikáció template : RESTful HTTP végpontok kialakítása és szolgáltatások kezelése kontrollerek segítségével.
- ASP.NET core Identity rendszert, ami az authozizációs és autentikáció feladatokat látja el.
- Swagger - ami lehetővé teszi a végpontok tesztelését

Entity framework: kapcsolat létrehozása az adatbázis és web api alkalmazás között

2.3. Frontend – React, React-Router-Dom, Vanta, JWT

A React egy modern JavaScript könyvtár, amelyet a felhasználói felületek fejlesztésére használnak. A komponens alapú megközelítés lehetővé teszi, hogy a felhasználói felület elemei újrafelhasználhatók és könnyen karbantarthatók legyenek. A React legnagyobb előnye a gyors renderelés, amely a virtuális DOM-nak köszönhetően gyors válaszidőt biztosít, különösen akkor, ha az alkalmazás komplex adatokat kezel. Emellett a React közösségi támogatása és ökoszisztémája (pl. React Router) segít a fejlesztés felgyorsításában.



A **React-Router-DOM** egy népszerű könyvtár a React alkalmazásokban, amely lehetővé teszi az ügyféloldali útvonalkezelést és a különböző komponensek közötti navigációt. Ezáltal egyoldalas alkalmazásokat (SPA) hozhatunk létre, ahol a felhasználók zökkenőmentesen navigálhatnak az oldalak között anélkül, hogy az egész oldal újratöltődne.

A **Vanta** egy JavaScript könyvtár, amely lenyűgöző 3D animált háttérhatások hozzáadását teszi lehetővé weboldalakhoz. React alkalmazásokban is könnyedén integrálható, így interaktív és látványos háttereket hozhatunk létre.

A **JWT (JSON Web Token)** egy nyílt szabvány (RFC 7519), amely lehetővé teszi a biztonságos adatátvitelt egy kliens és egy szerver között egy tömörített és aláírt token formájában. Webalkalmazásokban jellemzően **felhasználói hitelesítésre és jogosultság kezelésre** használják.

2.4. Frontend – WPF (Windows Presentation Foundation), HttpClient, Newtonsoft.Json, Xceed WPF Toolkit, JWT

A WPF (Windows Presentation Foundation) egy .NET-alapú grafikus felhasználói felületek létrehozására szolgáló keretrendszer, amelyet a Microsoft fejlesztett kifejezetten Windows platformra. A technológia lehetővé teszi, hogy a felhasználói felület XAML nyelv segítségével deklarativan legyen megírva, míg a mögöttes működés C# nyelven történik. Ez a felépítés támogatja az MVVM (Model-View-ViewModel) architektúrát, amely elősegíti a tiszta kódszerkezetet és a hatékony fejlesztést.



A projekt során a WPF-et egy natív adminisztrációs kliensalkalmazás megvalósítására használtuk. A felhasználói felület több nézetre tagolódik, például bejelentkezés, tranzakciók, helyszínek és anyagok kezelésére.

A WPF kliens a HttpClient osztály segítségével kommunikál a szerverrel. Ez a .NET beépített osztálya, amely lehetővé teszi REST API-k hívását HTTP protokollon keresztül. A **HttpClient** aszinkron működése biztosítja, hogy az alkalmazás felhasználói felülete ne fagyjon le a háttérben zajló adatlekérdezések vagy adatküldések során. Az API-k válaszait JSON formátumban dolgozzuk fel, amelyhez a széles körben elterjedt Newtonsoft.Json könyvtárat használjuk.

A kommunikáció biztonságát a **JWT (JSON Web Token)** alapú hitelesítés garantálja. A bejelentkezés során a felhasználó kap egy aláírt tokenet, amelyet minden további API-hívás mellé automatikusan csatol a program – ezzel biztosítva, hogy csak adminisztrátorok férhetnek hozzá a kliens funkcióihoz.

A felhasználói élmény fokozására az **Xceed WPF Toolkit** is integrálva lett, amely bővíti az elérhető vezérlők és stílusok körét. Az alkalmazás ezekből az alapvető komponenseket használja, például beviteli mezők vagy kiegészítő UI-elemek formájában.

A WPF technológia hatékony eszközként szolgált egy stabil, asztali adminisztrációs kliens kialakításához, amely modern felületen keresztül, biztonságosan és megbízhatóan kapcsolódik a backendhez.

2.5. Verziókezelés – Git/GitHub

A Git egy nyílt forráskódú verziókezelő rendszer, amelyet Linus Torvalds fejlesztett ki a Linux kernel fejlesztésének támogatására. Lehetővé teszi a fájlok különböző verzióinak nyomon követését, a módosítások dokumentálását és a csapatmunkát, akár offline

környezetben is. A Git gyors és megbízható, használható helyileg a saját gépeden vagy egy távoli szerveren keresztül, parancssoros és grafikus felületeken egyaránt.

A GitHub egy webalapú platform, amely a Git verziókezelő rendszerre épül. Lehetővé teszi a Git-adattárak (repositoryk) tárolását, megosztását és kezelését az interneten keresztül. A GitHub segítségével könnyedén együttműködhetsz más fejlesztőkkel, nyomon követheted a projektek előrehaladását, és hozzájárulhatsz nyílt forráskódú projektekhez.

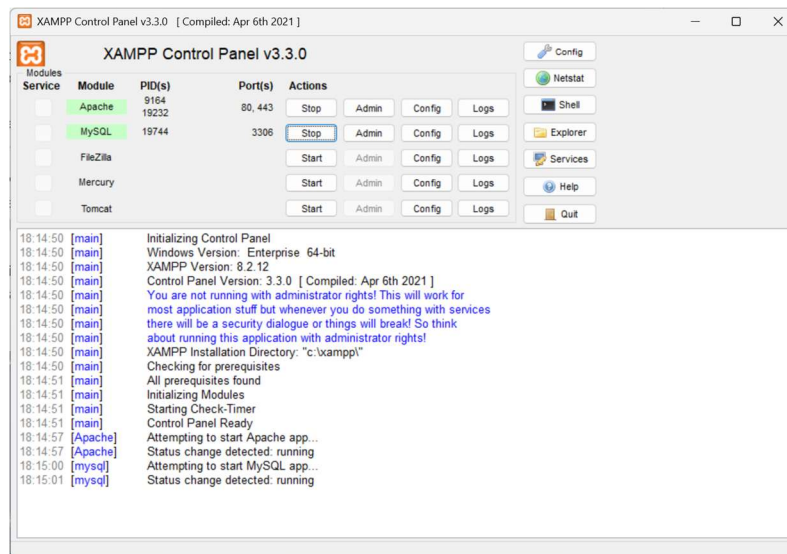
Fontos megjegyezni, hogy míg a Git egy verziókezelő szoftver, addig a GitHub egy szolgáltatás, amely a Gitet használja a kódok tárolására és kezelésére. Tehát a Git használatához nem feltétlenül szükséges a GitHub, de a GitHub a Git funkcionalitására építve kínál további szolgáltatásokat, mint például a kódmegosztás és az együttműködés támogatása.



3. Felhasznált fejlesztői környezetek bemutatása

3.1. Adatbázis - XAMPP (adatbázis host-olás)

A XAMPP egy olyan ingyenes, multiplatform keretrendszer, amely több szintén ingyenes webfejlesztéshez használt szoftvert képes futtatni létrehozva egy olyan környezetet amiben egyaránt megtalálható Apache web server, MySQL phpMyAdmin felülettel, FileZilla FTP szerver, stb , külön szolgáltatásonként indítva őket.

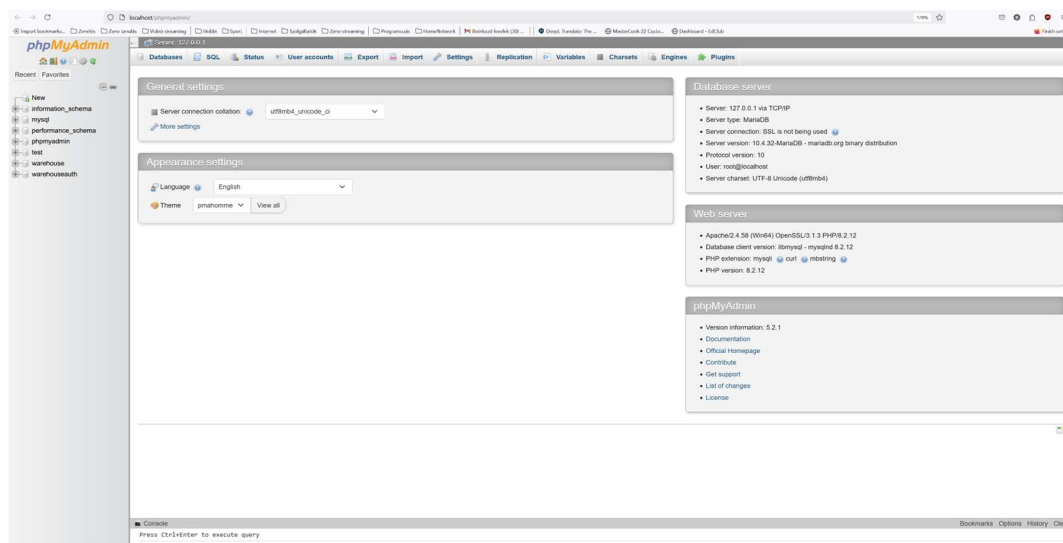


1. kép XAMPP indítókép

Elérhető szolgáltatások:

- Apache webservert - HTTP szerver szoftvert
- MySQL adatbázis szerver phpMyAdmin kezelőfelülettel: Alapvetően a MySQL csak egy konzolos kezelőfelülettel rendelkezik és különféle, saját parancsokon keresztül lehet használni. A phpMyAdmin ebben segít, egy webes böngészős felületen keresztül lehetővé teszi, hogy kényelmesen bonyolítsuk az adatbázis kezelés támasztotta feladatokat.(adatbázisok, táblák, kapcsolatok, triggerek létrehozása, manipulálása, exportálása, importálása, stb)
- FileZilla - FTP fájlserver
- Mercury - levelező szerver
- Tomcat - szintén HTTP szerver, de teljesen Java alapú web applikációnak biztosít saját környezetet.

A feladat megoldása során csak a Apache webserver és a MySQL+phpMyAdmin volt igénybe véve a számos szolgáltatás közül.



2. kép phpMyAdmin fő felülete

3.2. Backend – Visual studio

A Microsoft által fejlesztett és karbantartott Visual Studio integrált fejlesztői környezet (IDE) 1997-es megjelenése óta van velünk, több jelenleg használatos programnyelvet támogat, szorosan kötődik a Microsoft .Net fejlesztői környezethez, melynek számos tagjához tartalmaz specifikus eszközöket.

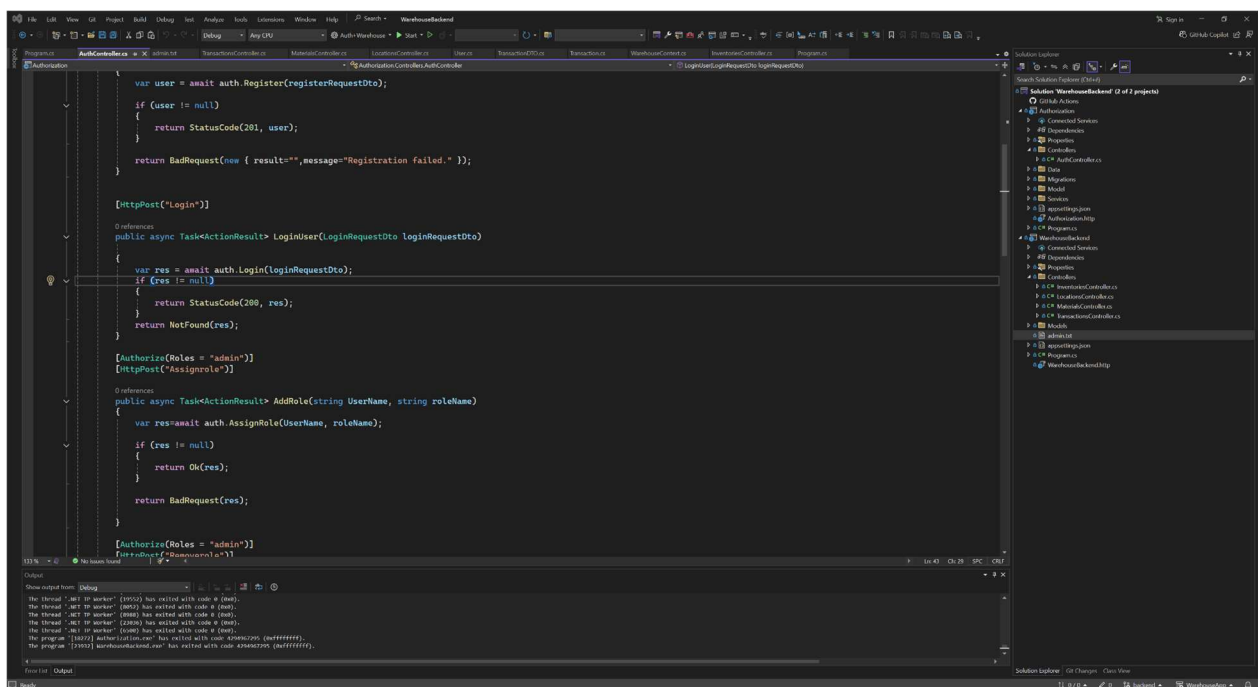
Jelenlegi aktuális verziója a Visual Studio 2022, 3 különféle kiadásban érhető el, amely verzióként egyre szélesedő eszközkészletével lefedi a közösségi programozók igényeitől egészen a kis -közepes és nagyvállalatokkal bezáróan.

A 3 kiadás:

- Community edition: ingyenes verzió, nem kereskedelmi forgalomba szánt szoftverek fejlesztéséhez, mint pl nyílt forráskódú, vagy oktatási, kutatási céllal készült szoftverekhez.
- Professional edition: igényli licenc vásárlását, kereskedelmi forgalomba kerülő szoftverek fejlesztését teszi lehetővé. Tudása is több mint a community edition-nek, például MSDN támogatást tartalmaz vásárolt license- től függően.
- Enterprise edition: Ez a verzió még további eszközöket tartalmaz a korábbi verziókhoz képest. Szintén kereskedelmi forgalomba kerülő szoftverek fejlesztését teszi lehetővé.

Főbb részei:

- kódszerkesztő: az alap, máshol is megszokott tartalmaz nyelvfüggetlen intellisense megoldást (kifejezések egyfajta automatikus kiegészítése kódolás közben az adott nyelvi szintaktikáknak megfelelően) , és újabban AI képességekkel lett kiegészítve amely tovább gyorsítja a kód írását azáltal hogy javaslatot tesz egész kód részek beírására.
- Debugger
- Tervezők (designer-ek): pl MS Form és WPF applikációk UI-jának tervezésre alkalmas, de tartalmaz ezeken felül több tervezőt is.
- Tesztelő eszközök
- egyéb eszközök: például tartalmaz NuGet csomagkezelőt, amelynek segítségével különböző, .Net keretrendszerhez készült szoftverkomponensekkel kiegészíthetjük programunk képességeit, így azokat már saját magunknak nem szükséges kifejleszteni. .



3. kép Visual Studio felülete

3.3. Frontend – Visual studio Code

A **Visual Studio Code (VS Code)** egy ingyenes, nyílt forráskódú, platformfüggetlen forráskód-szerkesztő, amelyet a Microsoft fejlesztett. Elérhető Windows, macOS és Linux operációs rendszereken, és célja, hogy egyesítse a kódszerkesztők egyszerűségét a fejlett fejlesztői eszközök funkcionalitásával. A Visual Studio Code célja, hogy egy könnyű, mégis

erőteljes eszközt biztosítson a fejlesztők számára, amely támogatja a modern webes és felhő alapú alkalmazások fejlesztését.

Főbb jellemzők:

- **IntelliSense:** Fejlett kódkiegészítési funkció, amely szintaktikai kiemelést és automatikus kódkiegészítést kínál, segítve a fejlesztőket a gyorsabb és pontosabb kódírásban.
- **Beépített Git integráció:** Lehetővé teszi a verziókezelést közvetlenül az editorból, megkönnyítve a kódváltozások nyomon követését és kezelését.
- **Bővíthetőség:** Számos kiterjesztés érhető el a Visual Studio Code Marketplace-en keresztül, amelyek további funkciókkal bővítik az editort, például új programozási nyelvek támogatásával vagy egyedi eszközökkel.
- **Beépített terminál:** Az editoron belül közvetlen hozzáférést biztosít a parancssorhoz, lehetővé téve a fejlesztők számára, hogy anélkül futtassanak parancsokat, hogy elhagynák az editort.
- **Debugging támogatás:** Lehetővé teszi a kód hibakeresését közvetlenül az editorból, támogatva a töréspontok beállítását, a változók figyelését és a kód lépésenkénti végrehajtását.

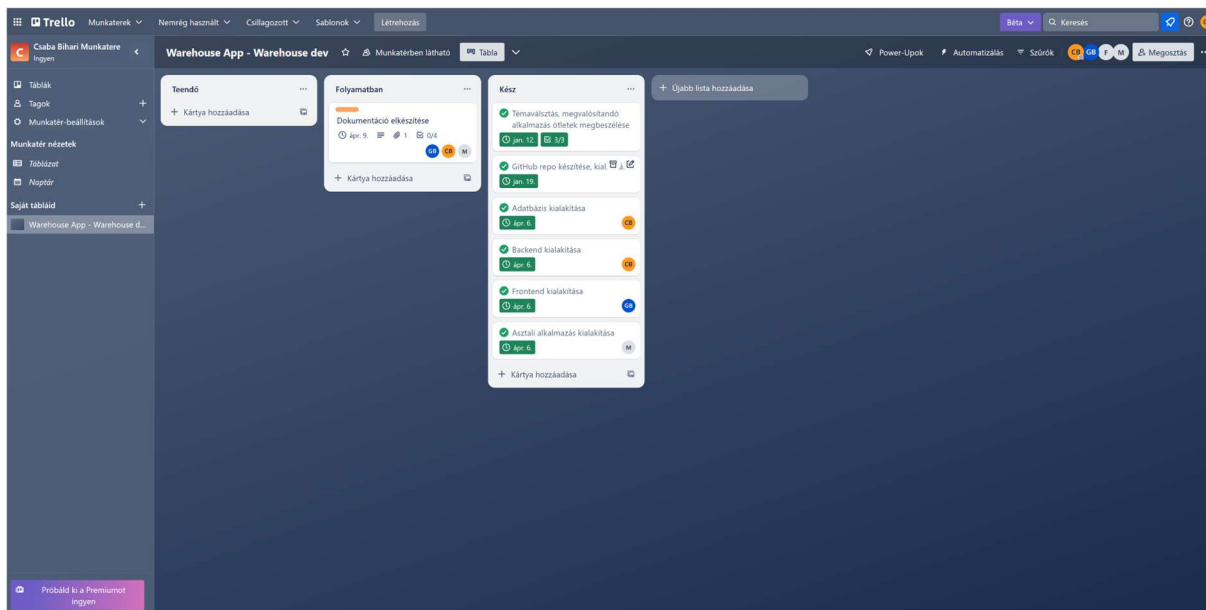


4. Felhasznált projektmanagement eszközök

4.1. Trello

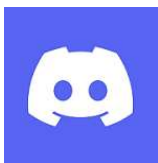
A Trello egy web alapú projektmanagement eszköz amelyen elősegíti mindenféle több résztvevős projektfeladat (nem csak szoftverfejlesztés) tervezését és lebonyolításának követését. Jelen világunkban rendkívüli módon felértékelődött a csapatmunka, mert a megoldandó feladatok egyre komplexebbekké váltak, valamint a megvalósítására rendelkezésre álló idő is egyre rövidebb.

Mivel webes alkalmazás lehetővé teszi, hogy a csapat tagjai földrajzilag különálló helyeken legyenek. A platformnak létezik ingyenes, korlátozott funkcionalitású változata (ezt használtuk jelen esetben) ami Kanban rendszer használatát teszi lehetővé egy tábla használatával, de kanban táblánkat több emberrel is megoszthatjuk, így kis projektek esetében ez is elég lehet. Az előfizetős verzió további, összetettebb projektmanagement eszközöket is tartalmaz, mint pl: több kártya, adatexportálás, stb.



4. kép Trello felülete

4.2. Discord



A Discord egy ingyenes, online kommunikációs platform, amely lehetővé teszi, hogy könnyedén tartsd a kapcsolatot barátaiddal, családdal vagy akár kollégáiddal. Hang-, videó- és szöveges üzenetek révén bármikor beszélgethatsz, oszthatsz meg tartalmakat, vagy éppen közösen játszhattok online. Eredetileg videojátékosok számára készült, hogy egyszerűbben kommunikálhassanak játék közben, de mára sokkal szélesebb körben használják. Különböző közösségek, baráti társaságok, tanulócsoporthok és vállalkozások is találják hasznosnak.

5. A szoftver bemutatása

5.1. Frontend, webes felhasználói felület működése

A **StorageDevs Warehouse Management System** kezdőoldala a felhasználót egy vizuálisan megkapó, de informatív üdvözlő képernyővel fogadja. Az oldal közepén egy áttetsző, kékes panelen helyezkedik el a bemutatkozó szöveg, amely összefoglalja az alkalmazás alapvető funkcióit és használati lehetőségeit. A háttérben egy logisztikai raktárról készített videó helyezkedik el (hang nélkül). A platform célja, hogy lehetővé tegye a felhasználók számára a készlet egyszerű és hatékony nyomon követését, szerkesztését és kezelését.

A kezdőoldal szövege tájékoztat arról is, hogy a rendszer különböző jogosultsági szinteket támogat:

- A **Materials** és **Locations** menüpontok lehetővé teszik az anyagok és tárolóhelyek hozzáadását, módosítását vagy törlését.
- Az **Inventory** menüpont a jelenlegi készletek megtekintésére szolgál.
- A **Transaction** menüpont pedig minden anyagmozgást dokumentál.

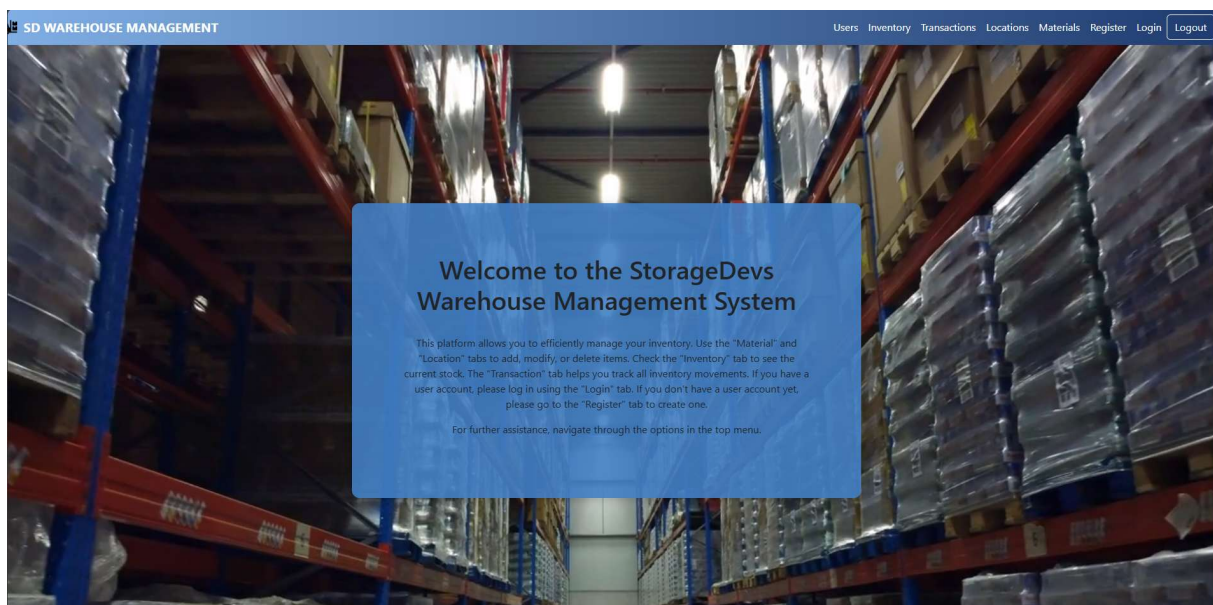
Amennyiben a felhasználó **Admin** jogosultsággal rendelkezik, elérhető számára a **Users** fül is, ahol új felhasználókat lehet regisztrálni vagy meglévőket módosítani és törölni.

Ez a bejelentkező rendszer lehetővé teszi a több szintű hozzáférés szabályozását: például a *User* nem törölhet anyagokat, míg a *Super User* már igen. Az *Admin* pedig a teljes rendszer felett rendelkezik, beleértve a felhasználók kezelését is.

A kezdőképernyő dizájnja egyszerre modern és funkcionális. A fejléc sávjában (Navbar) bal oldalon a rendszer logója és neve található, míg a jobb oldalon egy klasszikus menüsor sorakozik:

- **Inventory** - Összes készlet listázása
- **Transactions** - Tranzakciók
- **Locations** - Tárhelyek
- **Materials** - Anyagok
- **Register** - Regisztrációs felület
- **Login** - Bejelentkezési felület
- **Logout** - Kijelentkezés

A fejléc hátterét és kialakítását Bootstrap technológia segítségével készítettük el.

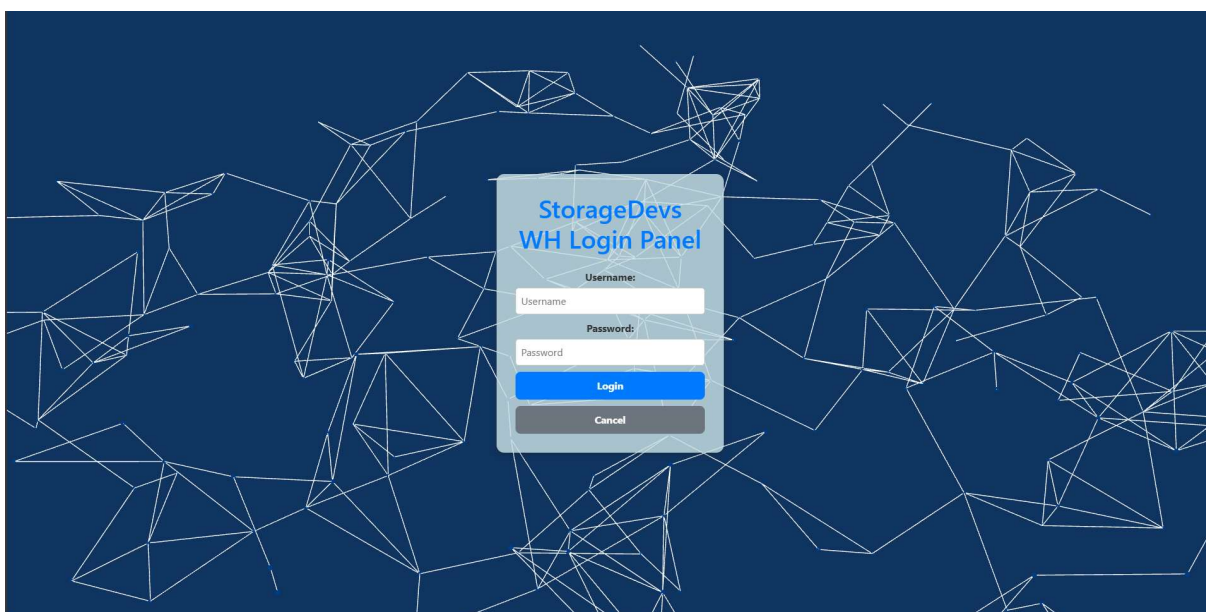


5. kép applikáció kezdőoldal

Ez az elrendezés gyors és átlátható navigációt biztosít a felhasználók számára. A felhasználó belépés után egyből láthatja, hogy mely funkciók elérhetők számára. Ez kulcsfontosságú a jogosultság-alapú működés szempontjából. A háttérként szolgáló videó vagy kép nem csupán díszítőelem: erőteljesen utal a rendszer valós alkalmazási területére, a logisztikai raktárak világára.

Bejelentkezés és kijelentkezés:

A belépési felület egy letisztult, modern megjelenésű login panelnek készült, amely a rendszer biztonságos használatának elsődleges kapujaként szolgál. A dizájn célja, hogy egyszerű és intuitív hozzáférést biztosítson a különböző felhasználók számára, miközben vizuálisan is illeszkedik a rendszer egészéhez.



6. kép bejelentkező oldal

A háttérben egy dinamikus, absztrakt animáció látható, amely vonalhálós pontkapcsolatokkal dolgozik. Ezt a Vanta könyvtár dinamikus 3D-s háttérének segítségével hoztuk létre. Ez egy modern webes esztétikát tükröz, miközben technológiai érzetet közvetít a felhasználó felé. A középen elhelyezkedő login panel áttetsző, világos háttérrel és kék árnyalatú gombbal rendelkezik.

A "**StorageDevs WH Login Panel**" elnevezésben a „WH” rövidítés a „Warehouse”, vagyis a raktár rövidítést takarja.

A rendszer validálja a felhasználónév és jelszó mezők tartalmát. Amennyiben ezek nem egyeznek egy érvényes felhasználói fiókkal, a felhasználót egy **hibajelzés** figyelmezteti:

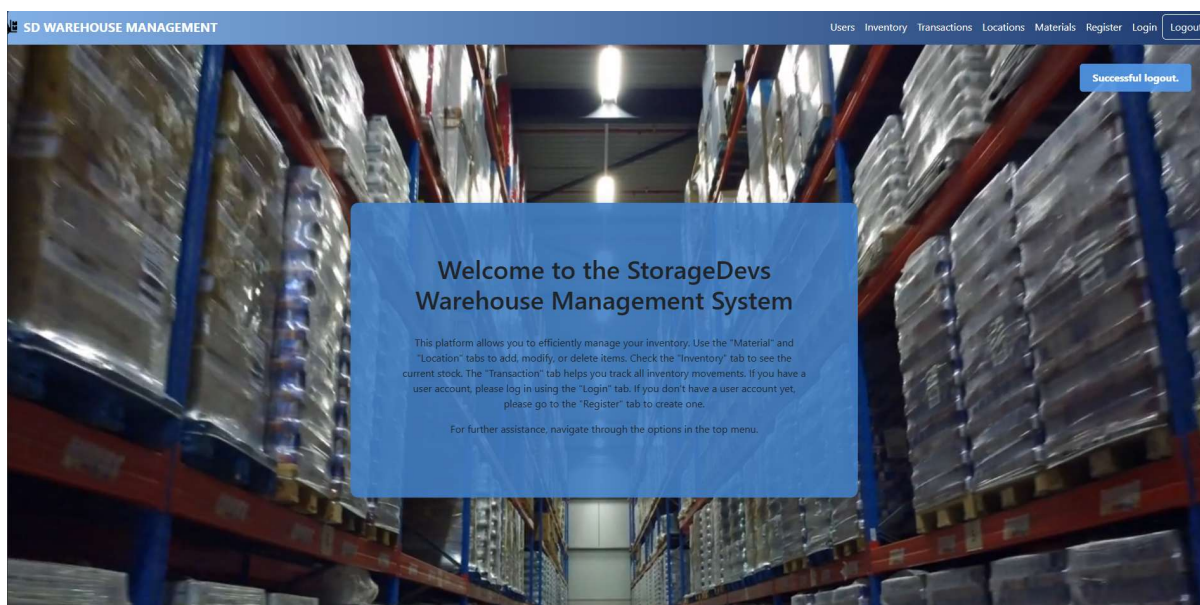
“Wrong username or password!”.

Ez a hibaüzenet **piros színnel** jelenik meg közvetlenül az űrlapmezők felett, jól látható módon. A hibakezelés célja, hogy azonnali visszajelzést adjon a felhasználónak, és lehetőséget biztosítson az adatok javítására. A bejelentkezési adatok ellenőrzése egy backend oldali API hívás során történik. A megadott hitelesítési adatok alapján a szerver ellenőrzi az adatbázisban szereplő felhasználókat, és amennyiben sikeres a belépés, egy **JWT (JSON Web Token)** kerül visszaküldésre, amely a felhasználói munkamenethez szükséges. Ez a token lehetővé teszi, hogy a felhasználó a rendszer többi részét elérje az engedélyei függvényében: pl. **User**, **SuperUser** vagy **Admin** szint szerint. A szintek részletes leírása a felhasználói felületi részen kerülnek bemutatásra.

A felhasználó a felső menüsávban található **Logout** gomb segítségével jelentkezhet ki a rendszerből. A sikeres kijelentkezést követően a képernyő közepén egy kis visszajelző mező jelenik meg, amely a következő szöveget tartalmazza:

„Successful logout.”

Ez az üzenet **3 másodperc** után automatikusan eltűnik, ezzel is biztosítva, hogy a felhasználó megerősítést kapjon a művelet sikerességéről, anélkül hogy bármilyen külön interakcióra lenne szükség.

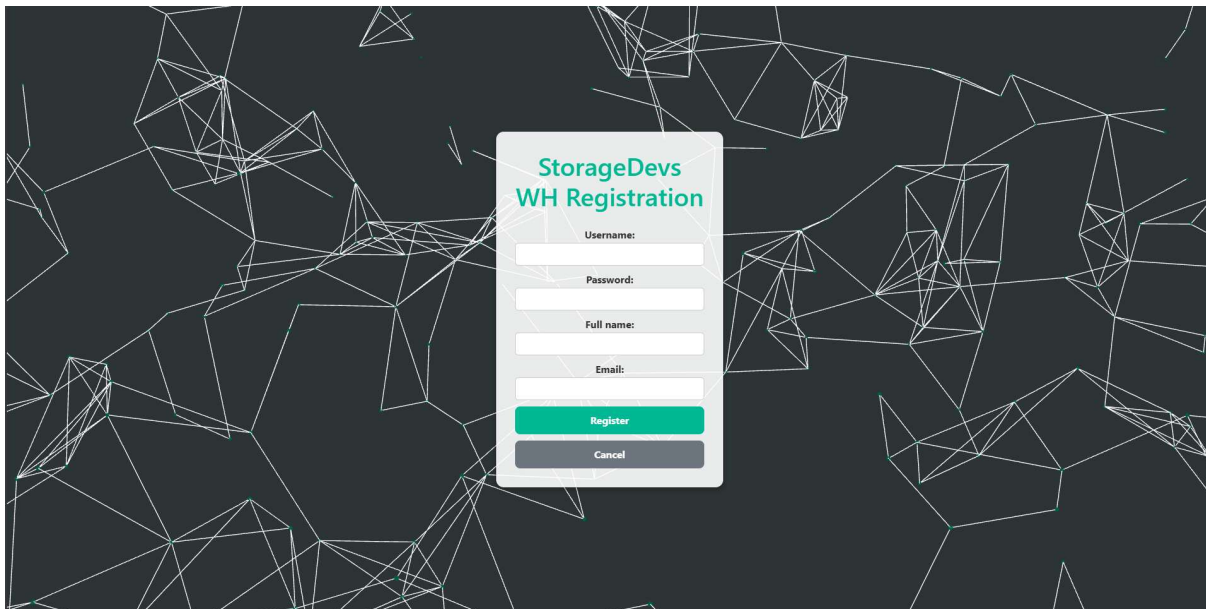


7. kép kijelentkezés

Regisztráció:

A rendszer regisztrációs felülete szintén egy átlátható, korszerű webes űrlap hozzáadott Vanta könyvtárral, mint ahogy a login panelnél is láthatjuk. A cím türkiz-zöld árnyalatú betűtípussal jelenik meg, amely vizuálisan elkülönül a mezők szürkés típusától. A felület három mezőt

tartalmaz a fiók létrehozásához: „Username”, „Password”, „Full name” és „Email”. Az űrlap alján található egy türkiz-zöld „Register” gomb, amely az adatok beküldését végzi.

The image shows a registration form titled "StorageDevs WH Registration". The form is white with a thin border and is centered on a dark gray background featuring a complex, white, geometric line pattern. The form contains four input fields: "Username:", "Password:", "Full name:", and "Email:". Below these fields are two buttons: a green "Register" button and a gray "Cancel" button.

8. kép regisztrációs oldal

A „Cancel” gombra kattintva vissza jutunk a kezdő oldalunkra a bejelentkezés és a regisztráció során is.

A regisztráció során többféle hiba is ellenőrzésre kerül:

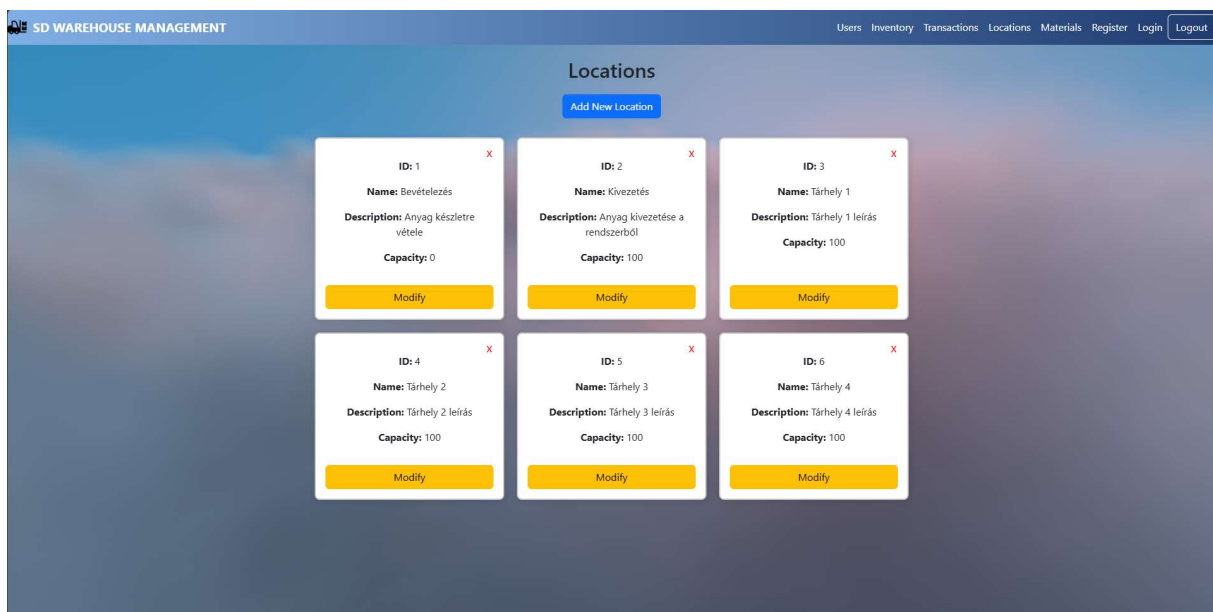
- Üres mezők esetén a rendszer figyelmeztet a mező kitöltésére.
- Ha a jelszó és a jelszó megerősítése nem egyezik, a következő angol nyelvű hibaüzenet jelenik meg a mezők fölött **piros** **színnel**:
"Passwords do not match"
- Ha a felhasználónév már foglalt, akkor szintén piros hibaüzenet látható:
"Username already taken"
- Ha minden egyezik, akkor a rendszer ezzel a szöveggel fog 2 másodperc múlva elnavigálni minket a login oldalunkra:
"Registration successful! Redirecting to login..."
- Ha egyéb probléma merül fel, akkor a rendszer az alábbi hibát fogja kiírni:
"Registration failed. Username might be taken."

- A regisztráció csak akkor lép életbe, ha az e-mail címbe szerepel a “@” karakter.

Felhasználói felület:

Locations:

A *Locations* oldal az alkalmazás azon felülete, amely a raktárban található különböző helyszínek/tárhelyek – például tárolók, be- és kivezetési pontok kezelését szolgálja. Az oldal célja, hogy a felhasználó átlátható módon megtekinthesse, módosítsa, törölje vagy újonnan felvegye a lokációs adatokat a rendszerbe.



9. kép elérhető lokáció listája

Az “Admin” és a “Superuser” felhasználók kétféle műveletet hajthatnak végre az egyes kártyákon:

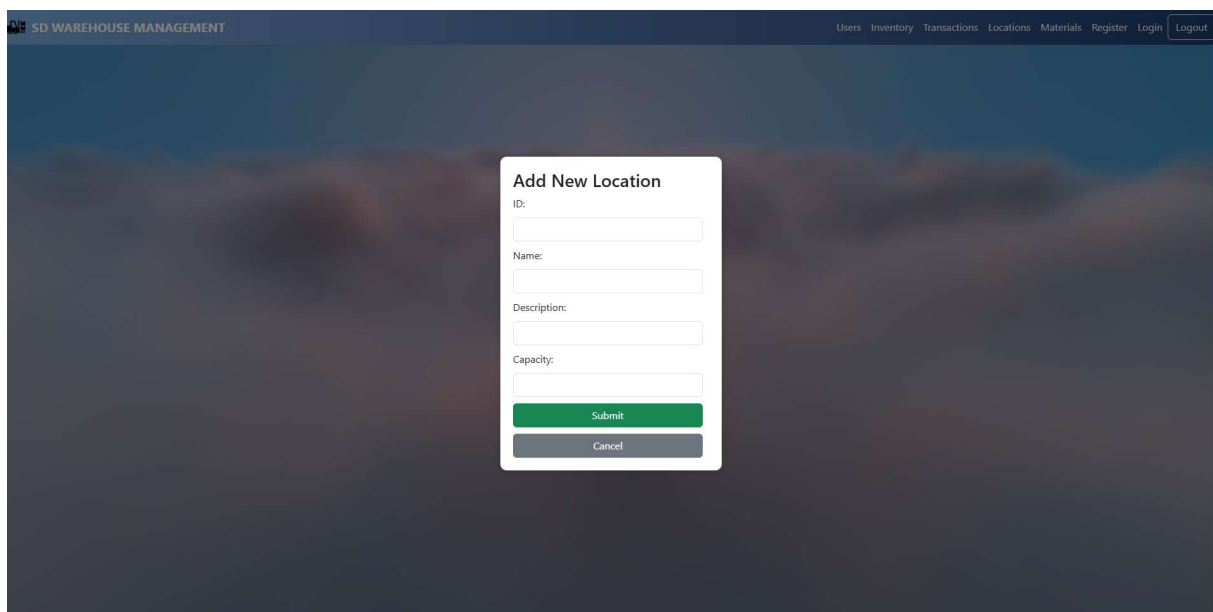
- **Modify gomb:** A kártya alján elhelyezett sárga színű gomb megnyomásával a kiválasztott helyszín adatai szerkeszthetők. Ez ugyan úgy, ahogy a hozzáadásnál is - egy formában történik, amely lehetővé teszi az értékek módosítását.
- **Törlés ikon (X):** A jobb felső sarokban található piros "X" ikon segítségével a kiválasztott lokáció véglegesen törölhető a rendszerből.

A felhasználók szerepköre befolyásolja az oldalak funkcionalitását:

- **User:** új anyagot rögzíthet, de nem törölhet és módosíthat
- **Super User:** az anyagokat módosíthatja és törölheti

- **Admin:** minden funkcióhoz teljes hozzáféréssel rendelkezik + felhasználókat módosíthat és törölhet

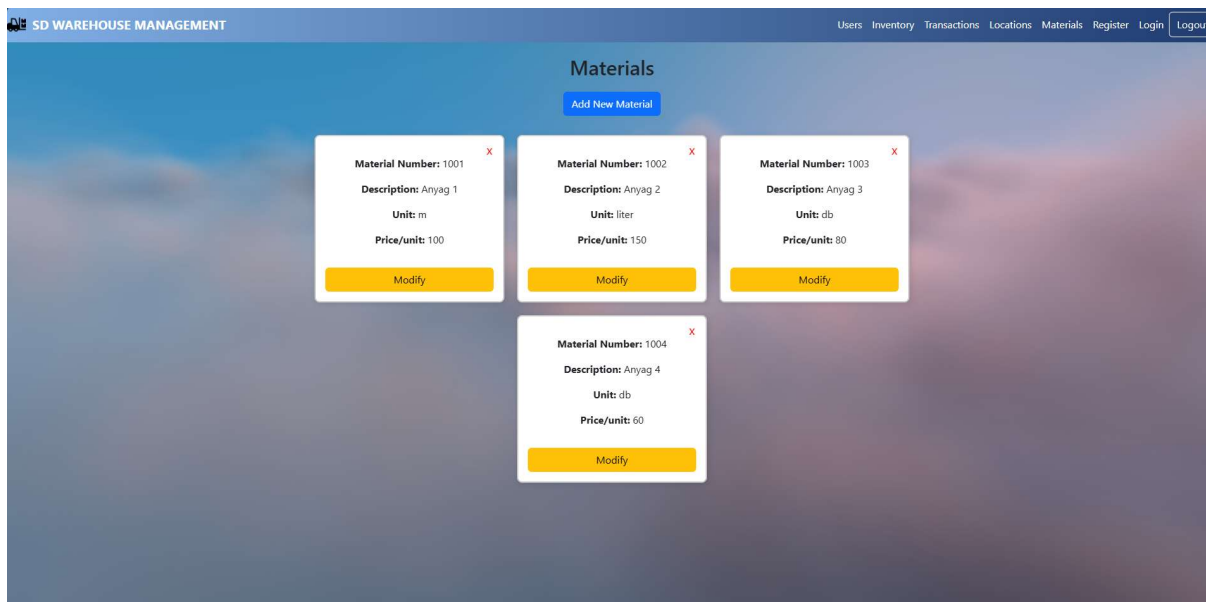
A "Locations" oldal cím közepén jelenik meg, alatta egy jól látható, *"Add New Location"* feliratú kék gomb található, amely új lokáció hozzáadására szolgál. Ennek megnyomásával egy felugró űrlap jelenik meg, ahol az új helyszín részletei (név, leírás, kapacitás stb.) adhatók meg. A háttér ezen esetben elsötétedik, hogy a felhasználó könnyen tudjon koncentrálni a beviteli mezőkre.

The image shows a web application interface for 'SD WAREHOUSE MANAGEMENT'. At the top, there is a navigation bar with links: Users, Inventory, Transactions, Locations, Materials, Register, Login, and Logout. The main content area is dark and blurred. In the center, a white modal form titled 'Add New Location' is displayed. The form contains four input fields: 'ID:', 'Name:', 'Description:', and 'Capacity:'. Below these fields are two buttons: a green 'Submit' button and a grey 'Cancel' button.

10. kép lokáció hozzáadása felület

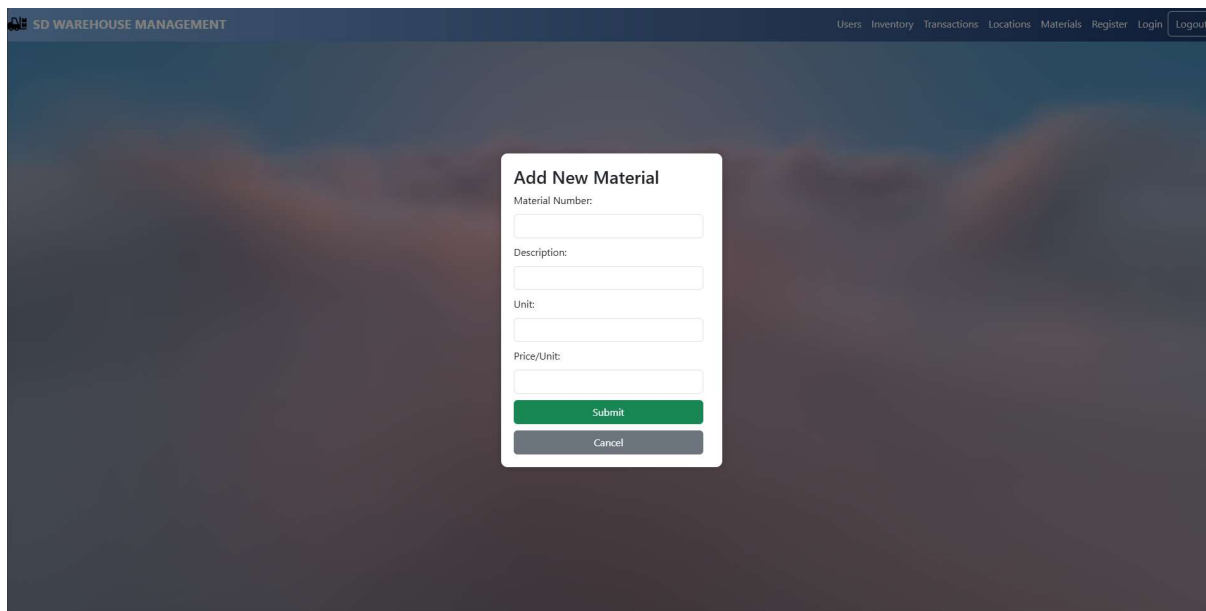
Materials:

A *Materials* komponens hasonló logikát követ, mint a *Locations* oldal. Az anyagok kezelése során a kártyák az adott anyag nevét, leírását, mennyiségét, mértékegységét és a hozzájuk rendelt tároló helyet jelenítik meg. Az interakciós elemek és a megjelenítési elvek megegyeznek, így a felhasználói élmény egységes marad az alkalmazás különböző részein.



11. kép elérhető anyagok listája

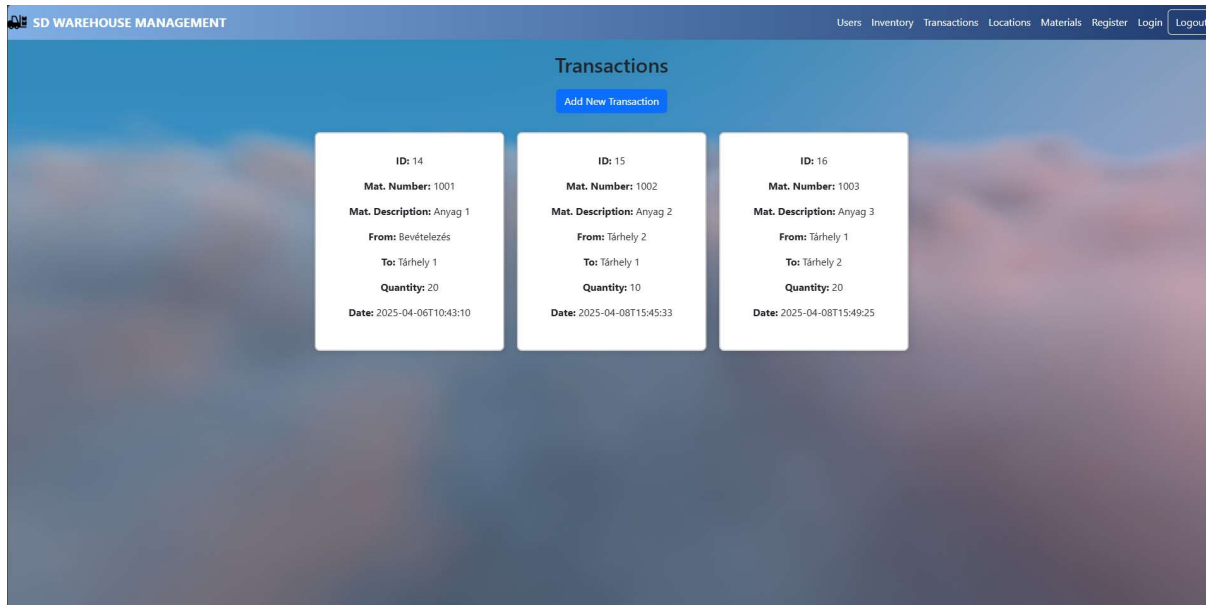
A *Materials* oldal az alkalmazás egyik központi felülete, amely az aktuálisan nyilvántartott anyagok kezelésére szolgál. Ez az oldal lehetőséget biztosít új anyagok rögzítésére, meglévők szerkesztésére vagy törlésére, továbbá jól strukturált módon jeleníti meg a nyilvántartásban szereplő bejegyzéseket. A hozzáadás a következőképpen mutatkozik meg az anyagoknál:



12. kép új anyag hozzáadása felület

Transactions:

A *Transactions* komponens a raktárkezelő rendszer egyik kulcsfontosságú része, amely az anyagmozgások naplózására szolgál. A felhasználók ezen az oldalon keresztül tudják rögzíteni, hogy mely anyag mikor, honnan hová és milyen mennyiségben került átmozgatásra. Ez biztosítja az átláthatóságot és a készletmozgások pontos nyomon követését.



13. kép tranzakciók listája

Az oldalon a felhasználók csak **új tranzakciókat tudnak hozzáadni**, meglévőket **nem lehet módosítani vagy törölni**. Ez a megkötés tudatos döntés, amely biztosítja az események naplózásának sérthetetlenségét, és megelőzi a visszamenőleges adatmanipuláció lehetőségét. Mivel a módosítás és törlés nem engedélyezett, így minden felhasználó, beleértve a Usereket is, **egyenlő hozzáféréssel rendelkeznek**.

SD WAREHOUSE MANAGEMENT

Users Inventory Transactions Locations Materials Register Login Logout

Add New Transaction

Mat. Number:

From (Location):

To (Location):

Quantity:

Submit

Cancel

14. kép új tranzakció hozzáadása felület

Az **"Add New Transaction"** gombra kattintva egy űrlap jelenik meg, amelyben megadható a tranzakcióhoz szükséges minden adat, beleértve az anyag nevét vagy számát, honnan és hová megy az anyag, valamint az tárolni kívánt mennyiséget.

A hozzáadásnál nem adjuk meg közvetlen az azonosítót és a dátumot, mivel automatikusan hozzáadja az adatbázis, így elkerülhetőek a hibák és biztosított az egységes adatkezelés.

A teljes felület reszponzív kialakítású, a kártyák rácsszerű elrendezésben jelennek meg, amelyek különböző képernyőméret esetén is optimálisan igazodnak a megjelenítő eszközhöz. A dizájn letisztult, modern hatást kelt, vizuálisan jól elkülönítve mutatja be az egyes elemeket, ezáltal könnyen kezelhető és átlátható marad nagy adatmennyiség esetén is.

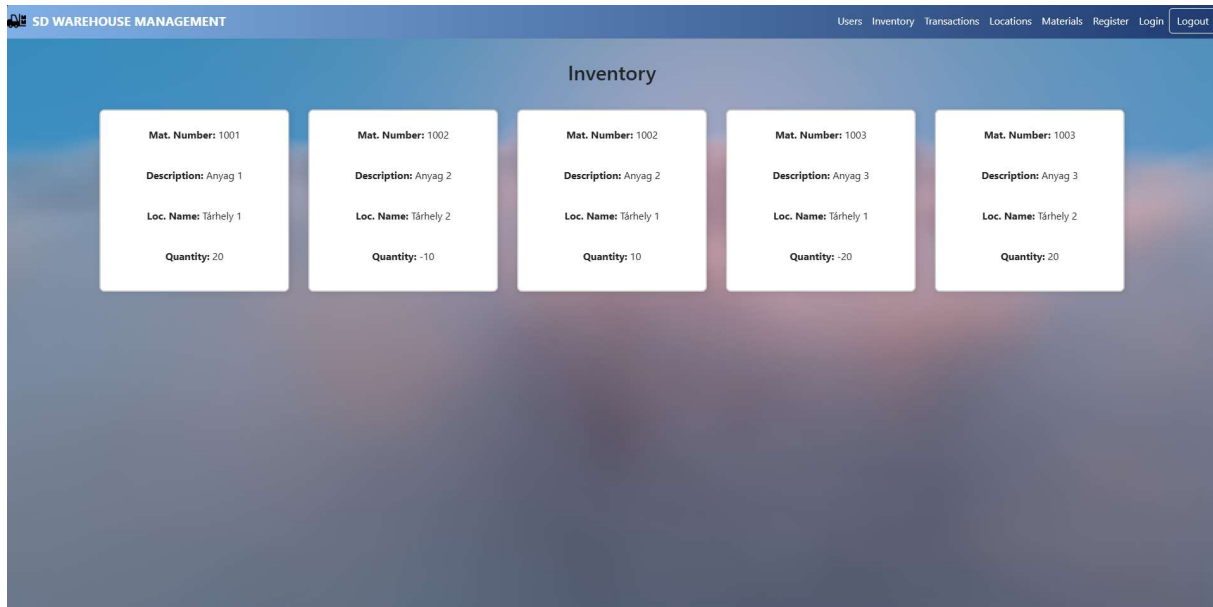
A felhasználói élmény vizuális fokozása érdekében az alkalmazás összes oldalán **Vanta**-t használunk dinamikus háttérként. A login és register oldalon valamint a kezdőlapon kívül a **Vanta.Clouds** effekt van beépítve az összes további oldalon, amely egy valós időben animált felhőmozgást jelenít meg, reagálva az egérmozgásra és a képernyőméretre.

A háttér beépítése **useEffect** hook-on keresztül történik komponens szinten.

A **THREE.js** könyvtárra épül, így szükséges a **three** csomag külön importálása is.

Inventory:

Az Inventory oldal célja az aktuálisan elérhető anyagkészletek vizuális, jól átlátható megjelenítése az egyes raktár helyeken. Ezen az oldalon a program kizárólag csak kiírja az adatokat és nem lehet további műveletet végrehajtani (hozzáadás, törlés, módosítás), mint a többi lapon.



15. kép készlet lista

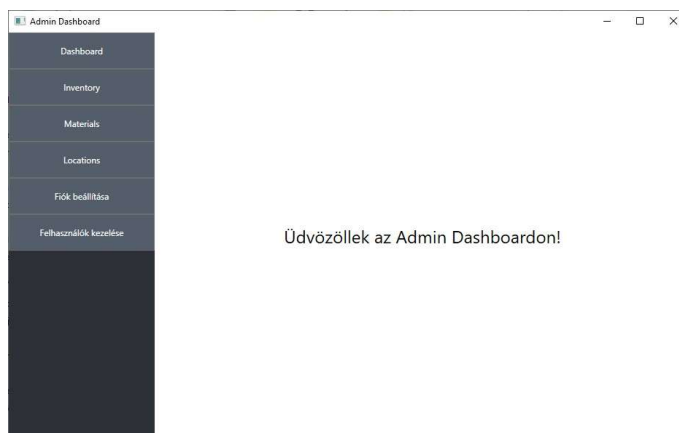
Az oldal az alábbi adatokat jeleníti meg kártyák formájában:

- **Mat. Number** – az anyag egyedi azonosítója
- **Description** – az anyag leírása
- **Loc. Name** – a tárolási hely neve
- **Quantity** – az adott anyag mennyisége az adott tárhelyen

Egy anyag több tárhelyen is tárolható, így akár ugyanaz az azonosító is megjelenhet többször, eltérő helyszínekkel.

5.2. Asztali kliens – WPF felhasználói felület működése

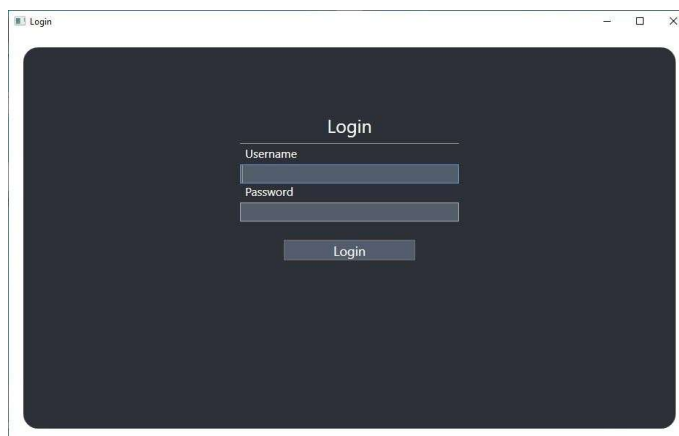
StorageDevs Warehouse Management System natív WPF-alapú asztali kliense egy olyan adminisztrációs célú alkalmazás, amelyet kizárólag a rendszerhez jogosultsággal rendelkező felhasználók – elsősorban adminok – használhatnak. A célja, hogy a backend API szolgáltatásait egy letisztult, gyors és stabil desktop alkalmazásba integrálja, így támogatva a készletkezelés, anyagmozgások és felhasználómenedzsment hatékony lebonyolítását.



A rendszer klasszikus ablakos felépítésű, a fő felület bal oldali menüoszlopból és egy dinamikusan változó tartalmi régióból áll. A felhasználó a bal oldali navigációs panel segítségével válthat a különböző funkciók között. A felület kialakítása egyszerű, modern, világos témájú, és célja a gyors eligazodás, a nagy adathalmazok könnyű kezelhetősége.

Bejelentkezés

A rendszerbe történő bejelentkezés egy külön ablakban történik. A felhasználó megadja a felhasználónevet és jelszavát, majd egy **„Bejelentkezés”** gombbal elindítja a hitelesítési folyamatot. A megadott adatok egy **HTTP POST** kérés formájában kerülnek továbbításra a backend felé, ahol a szerver **JWT (JSON Web Token)** alapú hitelesítést végez.



Sikeres hitelesítés esetén a szerver visszaküldi a tokent, amelyet a kliens eltárol, és minden további API-hívás során automatikusan csatol a kérés fejléceiben. Ez biztosítja a folyamatos, jogosultságokhoz kötött hozzáférést a rendszer különböző részeihez. A token lejáratí idejének figyelése és szükség esetén annak érvényesítése is a rendszer része.

Amennyiben a belépési adatok hibásak, a felhasználó egy piros színű figyelmeztető üzenetet kap: **„Hibás Felhasználónév, vagy jelszó!”**

Ez közvetlenül a beviteli mezők fölött jelenik meg jól látható módon. A hibaüzenet célja, hogy azonnali és egyértelmű visszajelzést adjon, és segítse a felhasználót az adatok javításában.

Sikeres bejelentkezés esetén a felhasználó átirányításra kerül a fő **Admin Dashboard** felületre, ahol az elérhető funkciók szerepkör szerint jelennek meg. A jogosultságokat a JWT token tartalmazza, és ezek alapján dinamikusan szabályozódik a felhasználói felület működése.

Főmenü (AdminDashboard)

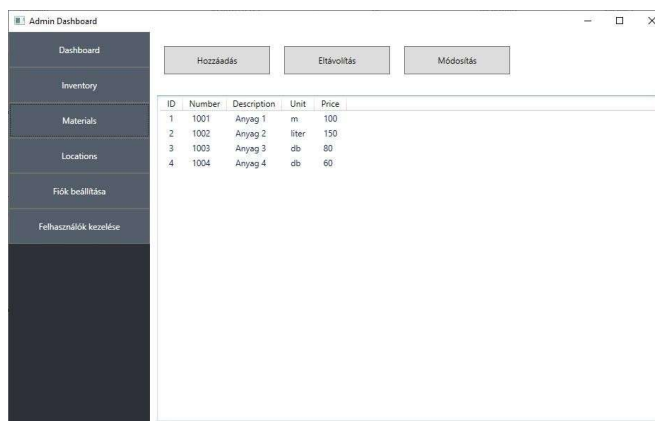
bejelentkezés után a felhasználó az **AdminDashboard** felületre kerül, amely egy oldalsó menüt és a középső tartalompanelt tartalmazza. Az oldalsáv lehetővé teszi a gyors navigációt a rendszer moduljai között:

- ❖ **Készlet (Inventory)** – aktuális anyagkészletek megtekintése, hozzáadása, mozgatása, törlése
 - **Tranzakciók (mozgatási előzmények)** – korábbi mozgások megtekintése
- ❖ **Anyagok (Materials)** – új anyag hozzáadása, meglévő módosítása vagy törlése
- ❖ **Lokációk (Locations)** – új tárhely hozzáadása, meglévő módosítása vagy törlése
- ❖ **Fiók beállításai (Settings)** – a felhasználói fiók kezelése
- ❖ **Felhasználók (Users)** – csak admin számára elérhető funkció, ahol új felhasználók rögzíthetők

Az oldalsáv mindig elérhető marad, így a navigáció gyors és kényelmes marad a felhasználó számára.

Materials – Anyagkezelés

Az **anyagok kezelése** oldal táblázatos nézetben jeleníti meg az összes nyilvántartott anyagot. Minden sor tartalmazza az anyag nevét, leírását, mennyiségét, mértékegységét és a hozzá tartozó tárolási helyet. Az adatok dinamikusan frissülnek a háttérben futó API-hívások révén, így mindig az aktuális állapot jelenik meg.



ID	Number	Description	Unit	Price
1	1001	Anyag 1	m	100
2	1002	Anyag 2	liter	150
3	1003	Anyag 3	db	80
4	1004	Anyag 4	db	60

A felület felső részén helyezkedik el az **„Hozzáadás”** gomb, amely egy külön ablakot nyit meg (**AddMaterialWindow**), ahol a felhasználó megadhatja az új anyag adatait.

A felület felső részén helyezkedik el az **„Eltávolítás”** gomb, amely egy igen/nem lehetőséggel az adott anyag összes adatait felsorolva, megerősítésre vár a kijelölt anyag vagy anyagok törlésére.

A felület felső részén található a **„Módosítás”** gomb, amely egy külön ablakot nyit meg (**EditMaterialWindow**) a kiválasztott sor(ok) szerkesztéséhez. Az ablakban az egyes mezők mellett egy **„Marad”** jelölőnégyzet található: ha ezt a felhasználó bejelöli, az adott mező inaktíválódik, és az értéke változatlan marad.

Új anyag hozzáadása

Anyag száma:

Leírás:

Mértékegység:

Ár:

Hozzáadás

Mégse

Megerősítés

Biztosan törölné ezt az anyagot?

Szám: 1002

Leírás: Anyag 2

Egység: liter

Ár: 150

Igen

Nem

Anyag módosítása

Anyagszám (jelenlegi):

1002

Új anyagszám:

☐ Marad

Leírás (jelenlegi):

Anyag 2

Új leírás:

☒ Marad

Mértékegység (jelenlegi):

liter

Új mértékegység:

☐ Marad

Ár (jelenlegi):

150

Új ár:

☐ Marad

Módosítás

Mégse

Locations – Raktárhelyek

tárhelyek kezelése (LocationsPage) oldal szinte azonos működésű, mint az anyagkezelés. Itt a felhasználó megtekintheti a meglévő adatokat, új lokációkat adhat hozzá, törölheti vagy módosíthatja azokat. Az adatok egy táblázatos nézetben jelennek meg, és tartalmazzák a helyszín nevét, leírását, kapacitását stb. Az adatok dinamikusan frissülnek a háttérben futó API-hívások révén, így mindig az aktuális állapot jelenik meg.

Admin Dashboard

Hozzáadás

Eltávolítás

Módosítás

ID	Name	Description	Capacity
3	Tárhely 1	Tárhely 1 leírás	100
4	Tárhely 2	Tárhely 2 leírás	100
5	Tárhely 3	Tárhely 3 leírás	100
6	Tárhely 4	Tárhely 4 leírás	100
7	Tárhely 5	Tárhely 5 leírás	100

Hasonlóan az anyagokhoz, itt is ugyanúgy úgy működik a három funkció.

Új Tárhely hozzáadása

Név:

Leírás:

Kapacitás:

Hozzáadás

Törölés megerősítése

Biztosan törölni szeretné ezt a helyszínt?

ID: 5

Név: Tárhely 3

Leírás: Tárhely 3 leírás

Kapacitás: 100

Igen

Nem

Lokáció módosítása

Név (jelenlegi):

Tárhely 3

Új név:

☐ Marad

Leírás (jelenlegi):

Tárhely 3 leírás

Új leírás:

☒ Marad

Kapacitás (jelenlegi):

100

Új kapacitás:

☐ Marad

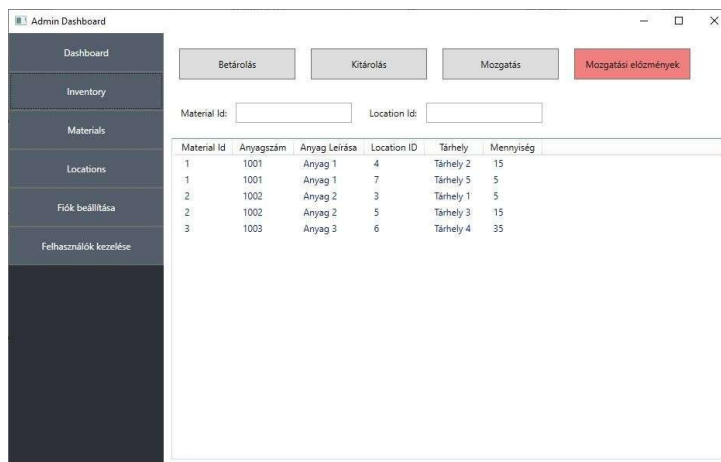
Módosítás

Mégse

Inventory – Készlet megtekintés és tranzakciók

Az **Inventory** célja az aktuálisan elérhető anyagkészletek vizuális, jól átlátható megjelenítése. Az adatok táblázatos nézetben jelennek meg, és tartalmazzák az anyag azonosítóját, leírását, a tárolási hely azonosítóját és nevét és a tárolt mennyiséget.

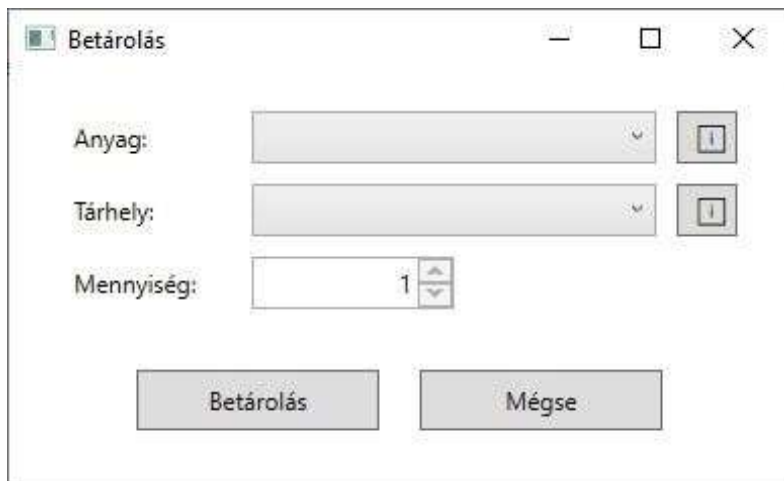
A felület felső részén helyezkedik el a **“Betárazás”, “Kitárazás”, “Mozgatás”** és a **„Mogazási előzmények”** gombok.



Material Id	Anyagszám	Anyag Leírása	Location ID	Tárhely	Mennyiség
1	1001	Anyag 1	4	Tárhely 2	15
1	1001	Anyag 1	7	Tárhely 5	5
2	1002	Anyag 2	3	Tárhely 1	5
2	1002	Anyag 2	5	Tárhely 3	15
3	1003	Anyag 3	6	Tárhely 4	35

Betárazás

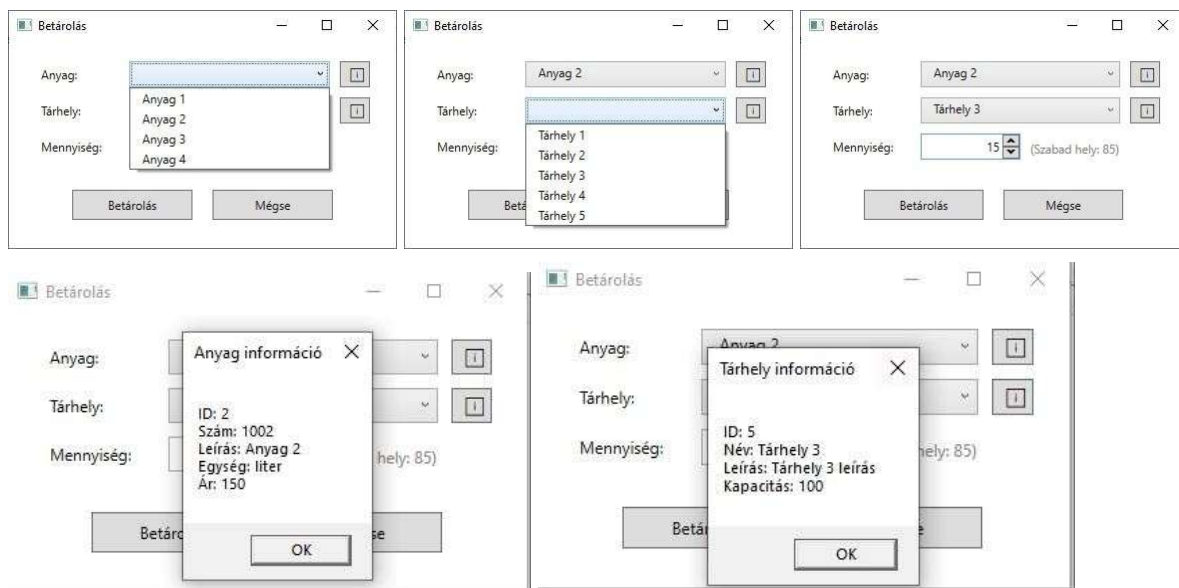
A felület felső részén található az **„Betárazás”** gomb, amely egy külön ablakot nyit meg, ahol egy lenyíló listából választhatjuk ki a már meglévő összes anyagokból és tárhelyekből. Mindkettő mellett információs gomb található, amely további részleteket (pl. azonosítókat, ár, tárhely kapacitás..stb, az összes azokhoz kapcsolódó adatokat) jelenít meg. A mennyiség numeric mezővel adható meg, ahol a rendszer automatikusan figyelembe veszi a tárhely szabad kapacitását – így csak 1 és a maximálisan még betárolható mennyiség közötti érték adható meg.



Anyag: 

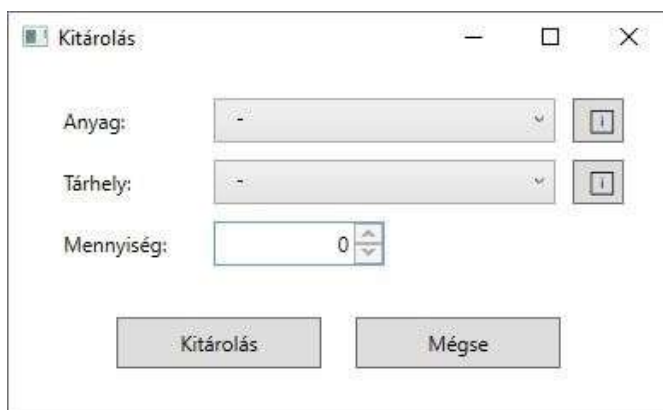
Tárhely: 

Mennyiség:  



Kitárazás

A „**Kitárazás**” gomb szintén külön ablakot nyit meg, ahol az anyag és tárhely kiválasztása dinamikusan egymásra épül: a rendszer csak azoknak a kombinációknak a kiválasztását engedi, amelyek valóban léteznek az adatbázisban. A mennyiség mező automatikusan igazodik az adott készlethez, és nem engedi túllépni az elérhető mennyiséget. Itt is információs gomb segíti a felhasználót mindkét mező mellett. Amennyiben a felhasználó előzőleg kijelölt egy vagy több sort az inventory táblázatban, és úgy nyomja meg a gombot, az ablak ennek megfelelően **automatikusan kitölti az adatmezőket** az adott készletegység adataival.



The 'Kitárolás' dialog box contains the following fields and buttons:

- Anyag:** Dropdown menu for material selection.
- Tárhely:** Dropdown menu for location selection.
- Mennyiség:** Spin box for quantity, with a label '(Tárhelyen: X)' indicating available stock.
- Buttons:** 'Kitárolás' (Allocate) and 'Mégse' (Cancel).

The screenshots show the following states:

- Initial state with 'Anyag' dropdown open, showing 'Anyag 1', 'Anyag 2' (selected), 'Anyag 3', and 'Anyag 4'.
- 'Anyag' set to 'Anyag 2', 'Tárhely' dropdown open showing 'Tárhely 1' and 'Tárhely 3'.
- 'Anyag' set to 'Anyag 2', 'Tárhely' set to 'Tárhely 3', 'Mennyiség' set to 1 (Tárhelyen: 15).
- 'Anyag' set to '-', 'Tárhely' set to 'Tárhely 5', 'Mennyiség' set to 0.
- 'Anyag' set to '-', 'Tárhely' dropdown open showing 'Anyag 1'.
- 'Anyag' set to 'Anyag 1', 'Tárhely' set to 'Tárhely 5', 'Mennyiség' set to 5 (Tárhelyen: 5).

Below the dialog boxes is the 'Admin Dashboard' window:

Admin Dashboard

Navigation menu: Dashboard, Inventory, Materials, Locations, Fiók beállítása, Felhasználók kezelése.

Buttons: Betárolás, Kitárolás, Mozcátás, Mozcátási előzmények.

Form fields: Material Id: [], Location Id: []

Material Id	Anyagszám	Anyag Leírása	Location ID	Tárhely	Mennyiség
1	1001	Anyag 1	4	Tárhely 2	15
1	1001	Anyag 1	7	Tárhely 5	5
2	1002	Anyag 2	3	Tárhely 1	5
2	1002	Anyag 2	5	Tárhely 3	15
3	1003	Anyag 3	6	Tárhely 4	35

Next to the dashboard is another 'Kitárolás' dialog box with 'Anyag' set to 'Anyag 3', 'Tárhely' set to 'Tárhely 4', and 'Mennyiség' set to 1 (Tárhelyen: 35).

5.3 Mozcátás

A „**Mozcátás**” gomb lehetőséget ad egy adott anyag egyik tárhelyről a másakra való áthelyezésére. Az anyag és a „honnan” mezők ugyanúgy dinamikusán szűrtek, mint a kitárazásnál, míg a „hová” mező automatikusan kizárja a kiindulási helyszínt. Mindhárom mező mellett információs gomb található. A mennyiség megadása során a rendszer figyeli a kiinduló készletet és a célhely szabad kapacitását, és csak az ezeknek megfelelő érték adható meg. Itt is igaz, hogy ha a felhasználó előzőleg kijelölt egy sort az inventory táblázatban, és úgy nyomja meg a gombot, akkor a felugró ablak **előre kitölti a mezőket** a kijelölt adat alapján.

Mozgatás

Anyag:

-

Honnan:

-

Hova:

-

Mennyiség:

0

Mozgatás

Mégse

Mozgatás

Anyag:

Anyag 2

Honnan:

Tárhely 1

Hova:

Tárhely 3

Mennyiség:

5

(készlet: 5, szabad hely: 85)

Mozgatás

Mégse

Mozgatás

Anyag:

-

Honnan:

-

Hova:

Anyag 1

Anyag 2

Anyag 3

Anyag 4

Mennyiség:

Mozgatás

Mégse

Mozgatás

Anyag:

Anyag 2

Honnan:

-

Hova:

-

Tárhely 1

Tárhely 3

Mennyiség:

Mozgatás

Mégse

Mozgatás

Anyag:

Anyag 2

Honnan:

Tárhely 1

Hova:

-

Mennyiség:

Tárhely 2

Tárhely 3

Tárhely 4

Tárhely 5

Mozgatás

Mégse

Admin Dashboard

Betárolás

Kitárolás

Mozgatás

Mozgatási előzmények

Material Id:

Location Id:

Material Id	Anyagszám	Anyag Leírása	Location ID	Tárhely	Mennyiség
1	1001	Anyag 1	4	Tárhely 2	15
1	1001	Anyag 1	7	Tárhely 5	5
2	1002	Anyag 2	3	Tárhely 1	5
2	1002	Anyag 2	5	Tárhely 3	15
3	1003	Anyag 3	6	Tárhely 4	35

Mozgatás

Anyag:

Anyag 3

Honnan:

Tárhely 4

Hova:

-

Mennyiség:

0

Mozgatás

Mégse

Mozgatási előzmények

A „**Mozgatási előzmények**” piros gomb megnyomására egy új ablak jelenik meg, amely az összes korábbi tranzakciókat listázza ki. A felhasználó így könnyedén nyomon követheti, mikor és hogyan történt az adott készletmozgás.

ID	Anyagszám	Leírás	Honnan	Hova	Mennyiség	Felhasználó	Dátum
5	1001	Anyag 1	Tárhely 2	Tárhely 5	5	superuser béla	4/9/2025 3:32:38 AM
4	1003	Anyag 3	Bevételezés	Tárhely 4	35	superuser béla	4/9/2025 3:30:18 AM
3	1002	Anyag 2	Bevételezés	Tárhely 1	20	user józsi	4/9/2025 1:20:24 AM
2	1002	Anyag 2	Tárhely 1	Tárhely 3	15	admin klára	3/11/2025 10:09:52 PM
1	1001	Anyag 1	Bevételezés	Tárhely 2	20	user józsi	3/11/2025 10:07:18 PM

Felhasználók kezelése

A **Users** felület kizárólag adminisztrátor jogosultságú felhasználók számára érhető el. A célja, hogy a rendszeren belüli jogosultságokat és felhasználói fiókokat központilag lehessen kezelni. Az oldal egy táblázatos nézetben jeleníti meg az összes regisztrált felhasználót.

A felület felső részén három gomb található: **„Hozzáadás”, „Módosítás”, „Törlés”**

ID	Username	Role
1	user józsi	user
2	superuser béla	superuser
3	admin klára	admin

Fiók beállítások (Settings)

A **Fiók beállítások** felület célja, hogy a felhasználók személyre szabhassák fiókjuk adatait. Ez a szekció szintén külön menüpontként érhető el az oldalsávból, és minden szerepkör számára elérhető.

Az oldal elsődleges funkciói:

- **Felhasználónév módosítása:** A mezők közvetlenül szerkeszthetők, és mentés után automatikusan frissülnek az adatbázisban
- **Jelszó módosítása:** A felhasználó megadhatja a jelenlegi jelszavát, majd kétszer az újat. A rendszer ellenőrzi, hogy az új jelszó megfelel-e a minimális biztonsági követelményeknek, és hogy a két mező megegyezik-e.

A dizájn követi a többi oldal struktúráját, esetleges visszajelzésekkel: siker, hiba üzenettel jelennek meg.

The screenshot shows the 'Admin Dashboard' interface. On the left is a sidebar with navigation links: Dashboard, Inventory, Materials, Locations, Frók beállítása, and Felhasználók kezelése. The main content area is titled 'Felhasználónév változtatása' (Username change). It contains a form with the following fields and buttons:

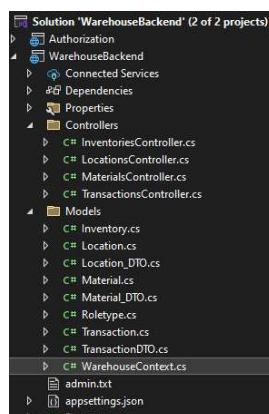
- Jelenlegi felhasználónév: **admin**
- Új felhasználónév:
- Új felhasználónév megismétlése:
- Felhasználónév változtatása (button)

Below this is the 'Jelszó változtatása' (Password change) section with the following fields and buttons:

- Jelenlegi jelszó:
- Új jelszó:
- Új jelszó megismétlése:
- Jelszó változtatása (button)

5.3. Backend

A szoftver backend része az ASP.NET 8-as verziójában készült Visual Studio fejlesztőkörnyezet segítségével. A WarehouseBackend "solution" két projektet tartalmaz: Authorization és WarehouseBackend. Mindkettő WEB API template alapú, ami alapvetően meghatározza, hogy vannak felépítve.



16. kép projekt struktúra

Warehouse backend projekt: adatbázissal való összekapcsolódást és annak menedzselését az Entity Framework segítségével végzi, amely úgy működik jelen esetben, hogy leképezi az adatbázis tábláit, mezőit, kapcsolatait C# osztályokká és objektumokká, amelyet a továbbiakban már LINQ segítségével (a C# saját lekérdező, "query" nyelve) közvetlenül tudunk kezelni, nincs szükség arra, hogy folyamatosan SQL lekérdezéseket alkalmazzunk. Ennek a gyakorlati formája a Models könyvtárba található WarehouseContext.cs osztály. A WarehouseContext osztály egy-egy példányát DI (Dependency Injection) segítségével példányosítjuk mikor szükség van rá.

A Model könyvtárban található egyéb, nem DTO-val végződő osztály egy egy adatbázis táblának rekordját reprezentálja, mint adatmodell.

A DTO osztályok feladat az, hogy a kliens szerver kommunikáció során a HTTP üzenetekben csak a szükséges információ kerüljön átadásra az üzenet törzsében. Erre látunk később példát.

Már csak a Controller könyvtárban szereplő Controller-re végződő osztályok ismertetése van hátra. Általánosságban elmondható hogy a kontrollerek felelnek a HTTP üzenet küldésért és fogadásáért. Az üzenet törzsébe JSON objektumként kerülnek a közölt információk. Nézzünk egy példát, a TransactionController osztályt, majd hogy ez hogyan is néz ki végpontként.

```
namespace WarehouseBackend.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class TransactionsController : ControllerBase
    {
        private readonly WarehouseContext _context;

        public TransactionsController(WarehouseContext context)
        {
            _context = context;
        }

        #region // GET: api/Transactions
        [Authorize(Roles = "user,superuser,admin")]
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Transaction>>> GetTransactions()
        {
            // GET: api/Transactions/5
            // POST: api/Transactions
        }
        #endregion
    }
}
```

17. kép osztály szerkezete

1. Az osztály attól lesz web api controller hogy szerepel benne az [ApiController] attribútum (amely controller specifikus viselkedéseket engedélyez, a [Route...] attribútum, amely megadja milyen URL útvonalon érhető el a controller és annak metódusai, valamint hogy az osztály a ControllerBase osztály egy leszármazottja ami lehetővé teszi az alapfunkciókat (HTTP válaszüzenetek pl.).
2. Szerepel még az osztályban a WarehouseContext beinjektálása konstruktoron keresztül, ami az adatbázis műveletek miatt fontos.
3. Itt jön az aktuális végpont definiálása. Fontos részei:
 - [Authorization...] attribútum, amely a végpont védelmét hivatott ellátni, és csak a zárójelben szereplő role-al rendelkező felhasználó képesek elérni. Ez elhagyható, de ekkor nem lesz védve, bárki küldhet rá kérést.
 - A [HttpGet] attribútum amely megadja, hogy milyen típusú kéréseket kezel ez a végpont.
 - Majd jön egy asszinkron módban (TASK) végrehajtott ActionResult típusú Gettransaction() függvény amit azt definiálja hogy mit tartalmazzon a válasz. Egy controller több végpontot is tartalmazhat, és a [HttpGet]-en kívül lehetőség van többfajta végpont létrehozása ([HttpPut], [HttpPost], stb)

Lássuk részletesebben ezt a függvényt:

```

public async Task<ActionResult<IEnumerable<Transaction>>> GetTransactions()
{
    try
    {
        var result = await
        (
            from transaction in _context.Transactions
            join material in _context.Materials
            on transaction.MaterialId equals material.MaterialId
            join locationFrom in _context.Locations
            on transaction.TransactionFromId equals locationFrom.LocationId
            join locationTo in _context.Locations
            on transaction.TransactionToId equals locationTo.LocationId
            select new GetAllTransaction...
        ).ToListAsync();

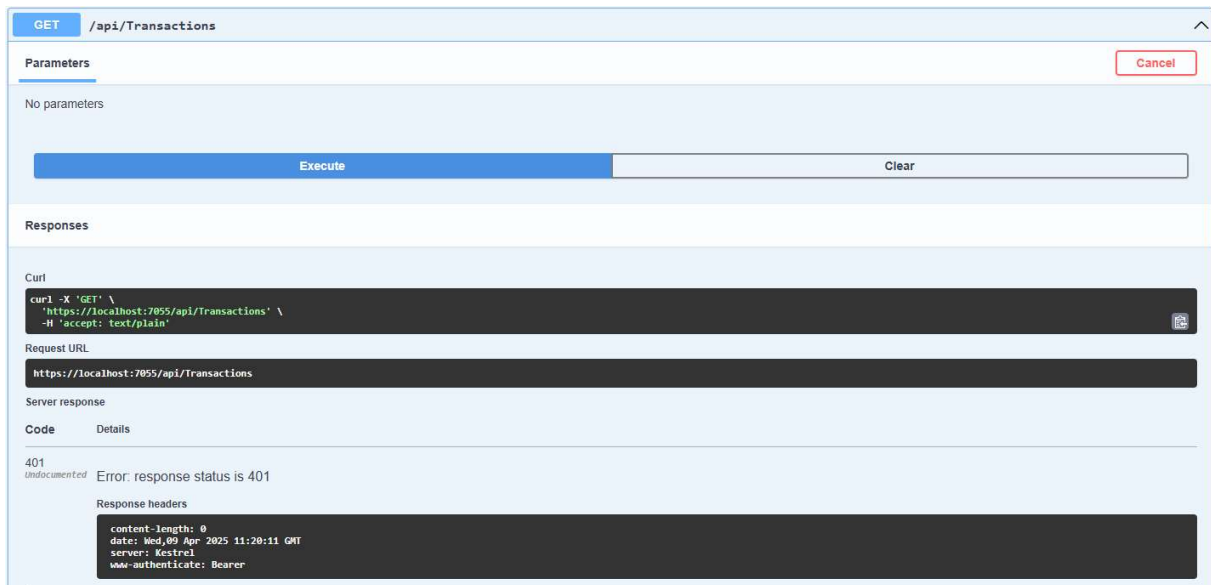
        return Ok(new { Result = result, message = "Successfull request" });
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { Result = "", Message = ex.Message });
    }
}

```

18. kép végpont szerkezete

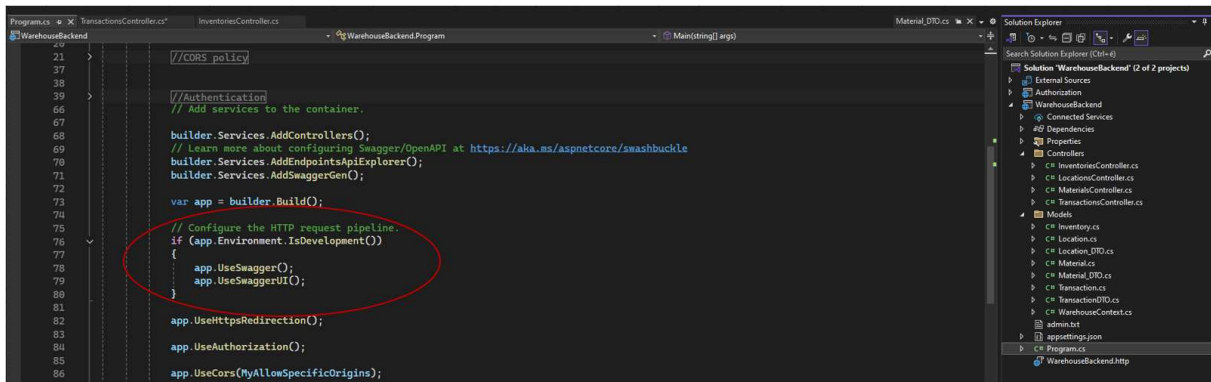
Ez ennek a függvénynek a feladata a transaction táblában lévő összes tranzakció adatainak összegyűjtése és a kliens számára történő elküldése. Tartalmaz egy LINQ [1] lekérdezést, amely az entity framework "táblák" összekapcsolásán keresztül egy lista objektumba, majd egy OK státuszkódú üzenetben, törzsében a json formátumra alakított listával válaszol. Amennyiben valamilyen kivétel történik a művelet során, akkor a 500-as státuszkódú, hibaüzenetet tartalmazó üzenetet küld vissza a kliens felé.

Hogy mindez hogyan néz ki a fogadó oldal szemszögéből azt a swagger nevű program segítségével lehet tesztelni. Ez küld egy üzenetet a megfelelő végpontra, és megjeleníteni a szervertől kapott választ.



19. kép swagger felület

A Swagger használata a ASP.NET 8 as verziójának része, a projekt program.cs osztályában kerül beállításra az alábbi képen látható módon, így tesztelhetjük a kialakított végpont viselkedését.



20. kép swagger hozzáadása projekthez

Az projekthez tartozó végpontok az alábbi képen láthatóak. Lényegében alap, CRUD műveleteket hajtanak végre a táblákon. Mindegyik végpont védett, és user role függő az elérhetősége. Az inventory csak GET műveletet tartalmaz, mert annak tartalma egy, a Transaction controller, POST művelete után végrehajtódó trigger módosítja.

Inventories		^
GET	/api/Inventories	▼
GET	/api/Inventories/CustomQuery	▼
Locations		^
GET	/api/Locations	▼
POST	/api/Locations	▼
GET	/api/Locations/{id}	▼
PUT	/api/Locations/{id}	▼
DELETE	/api/Locations/{id}	▼
Materials		^
GET	/api/Materials	▼
POST	/api/Materials	▼
GET	/api/Materials/{id}	▼
PUT	/api/Materials/{id}	▼
DELETE	/api/Materials/{id}	▼
Transactions		^
GET	/api/Transactions	▼
POST	/api/Transactions	▼
GET	/api/Transactions/{id}	▼

21. kép létrehozott warehouse végpontok

Authentication projekt

A Microsoft Identity csomag felhasználásával készült. Feladata az autentikáció (felhasználó azonosítása, regisztrálás, jelszó útján) valamint az autorizáció. (milyen részeit jogosult a programnak a használni az adott felhasználó). Ebben a projektben (vagyis a hozzátartozó adatbázisban) kerül tárolásra a felhasználó adatai (pl. jelszó megfelelő titkosítással), jogosultságai, valamint ez felel a belépés után előállítani azt a JWT token, aminek birtokában az adott felhasználó elérheti a felhasználói szerepköréhez tartozó végpontokat. Ebben a projektben található végpontokon keresztül lehet a felhasználó managementet végrehajtani. A definiált végpontok:

Auth		^
POST	/auth/Register	▼
POST	/auth/Login	▼
POST	/auth/Assignrole	▼
POST	/auth/Removerole	▼
DELETE	/auth/DeleteUser	▼
GET	/auth/GetAllUser	▼

22. kép létrehozott autentikációs végpontok

A projektekben felhasznált csomagok:

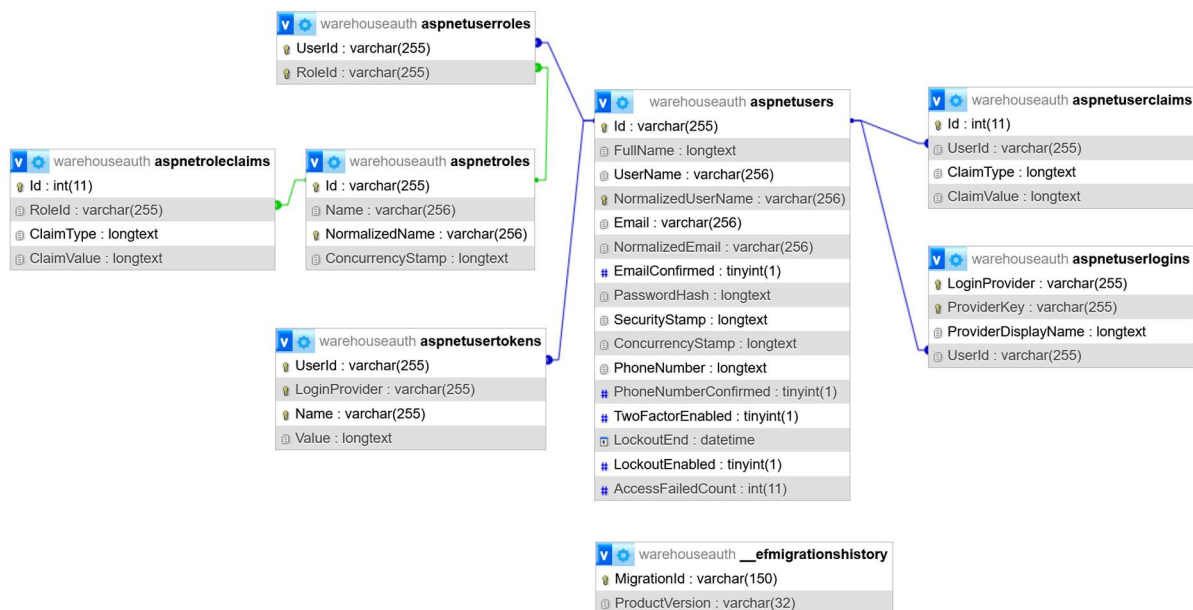
	Microsoft.AspNetCore.Authentication.JwtBearer by Microsoft ASP.NET Core middleware that enables an application to receive an OpenID Connect bearer token.	8.0.14 9.0.4
	Microsoft.AspNetCore.Identity.EntityFrameworkCore by Microsoft ASP.NET Core Identity provider that uses Entity Framework Core.	8.0.14 9.0.4
	Microsoft.EntityFrameworkCore.Tools by Microsoft Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.	8.0.13 9.0.4
	Microsoft.VisualStudio.Web.CodeGeneration.Design by Microsoft Code Generation tool for ASP.NET Core. Contains the dotnet-aspnet-codegenerator command used for generating controllers and views.	8.0.7 9.0.0
	MySQL.EntityFrameworkCore by Oracle Corporation MySQL.EntityFrameworkCore adds support for Microsoft Entity Framework Core.	8.0.11 9.0.0
	Swashbuckle.AspNetCore by domaindrivendev Swagger tools for documenting APIs built on ASP.NET Core	6.6.2 8.1.0

23. kép felhasznált csomagok

5.4. Adatbázis

Az alábbiakban a szoftver adatbázis oldala kerül bemutatásra. Nem cél a technikai részletek ismertetése, sokkal inkább az adatbázis szerkezete és funkciójának megismerése.

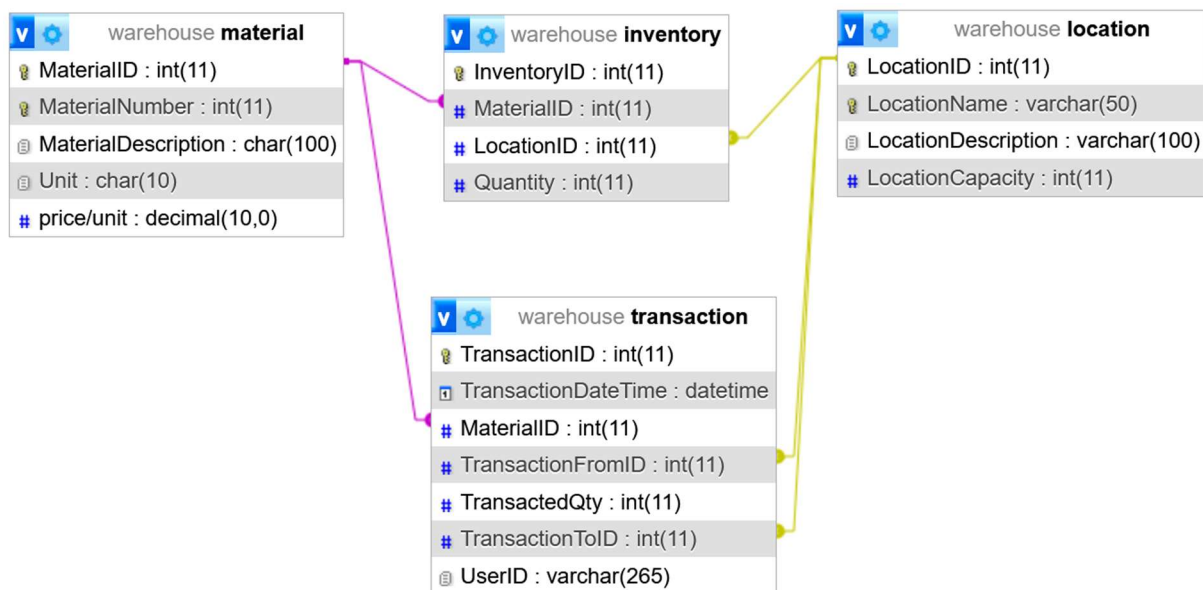
A szoftver a gyakorlatban két adatbázist tartalmaz. Az egyik adatbázis magát a raktározási műveletekkel kapcsolatos adatokat tartalmazza (warehouse.sql), míg a másik az ASP.NET core Identity nevezetű autorizációs és autentikációs moduljának adatbázisát (warehouseauth.sql) , amelyben a felhasználói accountok és jogkörök és tokenek kerülnek tárolásra. Mindkét adatbázis MySQL adatbázis kezelő rendszeren fut. Először a felhasználó menedzser adatbázist mutatom be néhány szóban.



24. kép autorizációs adatbázis szerkezete

Az adatbázis szerkezetét maga az Identity modul határozza meg, de lehetőség van arra, hogy az *AspNetUsers* táblát, - amely a felhasználói attribútumokat tartalmazza - saját attribútumokkal egészítsük ki. Az adatbázist Entity Framework-el hozza létre és menedzseli az Identity modul megfelelő osztálya.

A korábban már említett másik adatbázis, ami magát a raktározási feladatot látja el a következő megfontolások szerint lett kialakítva.



25. kép warehouse adatbázis szerkezete

Az adatbázis nem bonyolult, mindössze 4 tábla alkotja. Itt a cél egy olyan alap kialakítása volt, amely a legalapvetőbb funkciókat tartalmazza, miszerint: van valaminak tudni szeretnénk a hollétét, mennyiségét, azt hogy mikor került oda mekkora mennyiségben, és ki rakta oda. Képesnek kell lennie kezelni azt is, hogy ha valamit átrakunk valahova máshova. Valamint ha szeretnénk tudni, hogy hol miből mennyi van éppenséggel.

A “valamit” a *material* tábla reprezentálja, amelyben definiálhatunk anyagjellemzőket a tábla mezőinek keretein belül. A tábla mezőinek nevei magukért beszélnek, talán a *Unit* mezőt érdemes elmagyarázni. Ez az adott anyag mértékegysége, pl: darab, m³, kg, lehet.

MaterialID	MaterialNumber	MaterialDescription	Unit	price/unit
1	1001	Anyag 1	m	100
2	1002	Anyag 2	liter	150
3	1003	Anyag 3	db	80
4	1004	Anyag 4	db	60

26. kép material tábla példa tartalom

A “valahová”-t a *location* tábla reprezentálja, ahol szintén magunk tudunk különféle “tárhelyeket” definiálni. A ki- és bevételezés egy-egy tárhelyként van definiálva.

LocationID	LocationName	LocationDescription	LocationCapacity
1	Bevételezés	Anyag készletre vétele	0
2	Kivezetés	Anyag kivezetése a rendszerből	0
3	Tárhely 1	Tárhely 1 leírás	100
4	Tárhely 2	Tárhely 2 leírás	100

27. kép location tábla példa tartalom

A tárolás folyamatát a *transaction* tábla reprezentálja. Itt kerülnek tárolásra az anyagmozgatás részletei.

TransactionID	TransactionDate Time	MaterialID	TransactionFromID	TransactedQty	TransactionToID	UserID
1	2025-04-06 10:43:10	1	1	20	3	06de4a98-4eb5-42f7-b3b1-8275ec220b88

28. kép transaction tábla példa tartalom

Az aktuális készletet pedig az inventory tábla tartalmazza. Miből éppen mennyi van. Ennek a táblának a módosításáért egy trigger felel, amely a transaction adatoknak megfelelően módosítja a készletet. (módosít meglévő rekordot , hozzáad rekordot ha nincs még olyan anyag-lokáció páros a táblázatba amit módosítani lehetne)

InventoryID	MaterialID	LocationID	Quantity
1	1	3	20

29. kép inventory tábla példa tartalom

6. Irodalomjegyzék

[Mi az a MySQL, és hogyan használják általánosan a webfejlesztésben? - EITCA Akadémia](#)

[Tutoriál: Bevezetés a Reactbe – React](#)

[\[Csapatmunka, Git-el\] 1. rész: Mi is az a Git? - INTO](#)

[Git alapok - Kezdőknek és haladóknak](#)

[Learn - Git - 1. Elmélet](#)

[Router - Korom Richárd oktatás](#)

[vanta - npm](#)

[What is Visual Studio Code?](#)

[Discord: a szórakozás és együttműködés platformja - Modern Háziasszony](#)

[Get Started with JSON Web Tokens](#)

7. Ábra jegyzék

1. kép XAMPP indítókép	6
2. kép phpMyadmin fő felülete	7
3. kép Visual Studio felülete	8
4. kép Trello felülete.....	10
5. kép applikáció kezdőoldal	11
6. kép bejelentkező oldal.....	12
7. kép kijelentkezés	13
8. kép regisztrációs oldal	14
9. kép elérhető lokáció listája.....	15
10. kép lokáció hozzáadása felület.....	16
11. kép elérhető anyagok listája.....	17
12. kép új anyag hozzáadása felület.....	17
13. kép tranzakciók lisája.....	18
14. kép új tranzakció hozzáadása felület.....	19
15. kép készlet lista	20
16. kép projekt struktúra.....	29
17. kép osztály szerkezete	30
18. kép végpont szerkezete.....	31
19. kép swagger felület	32
20. kép swagger hozzáadása projekthez.....	32
21. kép létrehozott warehouse végpontok	33
22. kép létrehozott autentikációs végpontok	33
23. kép felhasznált csomagok	34

24. kép autorizációs adatbázis szerkezete	35
25. kép warehouse adatbázis szerkezete	36
26. kép material tábla példa tartalom.....	36
27. kép location tábla példa tartalom	37
28. kép transaction tábla példa tartalom.....	37
29. kép inventory tábla példa tartalom.....	37

8. Mellékletek

8.1. Github link

<https://github.com/StorageDevs/WarehouseApp.git>

8.2. Trello link

<https://trello.com/invite/b/67f0d529bd7de4dd75fc7455/ATTIb02932fcaf97548c041ca41590c784912905EBE0/warehouse-app-warehouse-dev>